Index

SRNO	Topic Name
1	Introduction
2	Types of database
3	Types of SQL command a. DDL b. DML c. DQL d. DCL e. TCL
4	Basic select query a. Where b. Logical operator AND,OR,NOT c. IN,BETWEEN d. like e. Aggregate function f. Group by g. Having h. Order by
5	Constraints a. Not null b. Unique c. Primary d. Foreign e. Check f. Default g. On delete cascade h. On update cascade i. On delete set null j. Auto_increment
6	Keys a. Primary b. Unique c. Foreign d. Candidate e. Super key f. Surrogate key g. Composite key h. Composite unique i. Composite foreign
7	Relationship and its practical implementation a. One to one b. One to many c. Many to one d. Many to many

8	**Joins** **a. Inner join** **b. Left join** **c. Right join** **d. Outer join** **e. Cross join**
9	**Normalization** **a. 1st NF** **b. 2nd NF** **c. 3rd NF** **d. 3.5 NF** **e. 4th NF**
10	**Subquery** **Views** **Index** **Procedure** **Trigger** **Function** **Cursor** **Union, intersection ,difference operation** **Acid properties**

Q. What is SQL?

SQL stands for structure query language. Basically it is programming language work with relational database management systems.
Before work with SQL we need to know the following things

a. **What is database & Important of database?**

   Database is a technique where we can organize your data internally or provide relationship between data as well as store data in particular structure and retrieve data according to user request called as database

   Q. Why use databases or What is the importance of databases?

   1. **Provide permanent storage to data:** means using a database we can store user data permanently in database so user can user stored data in future as per need
      Example: Suppose consider if we think about techhub application we store student data as admission permanently in our techhub database for maintain record of student

   2. **Provide security to data:** means no one can access user data directly

We can give the authentication to user as well as give authorization to user data

Example: suppose we have DBA in our system and we have some normal user in system so DBA can delete or update data but user can view data we can set this type
Of privileges on database

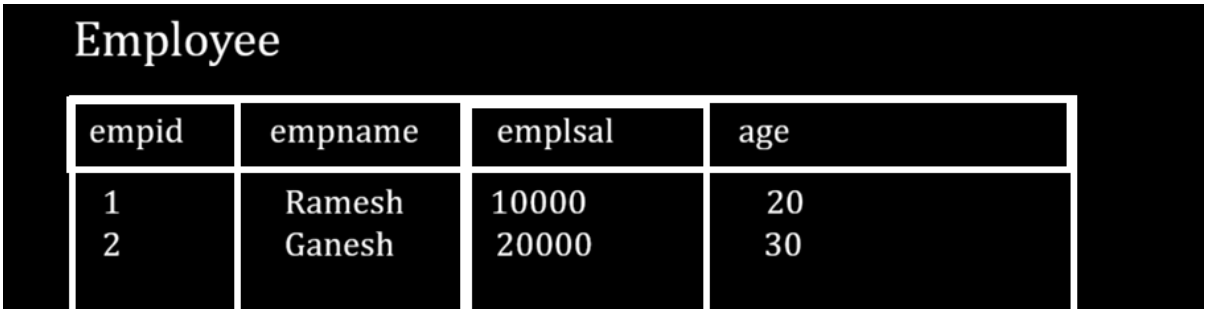
3. **Provide retrieval as per user request :** means we can fetch data according to user request like as suppose we want to fetch employee whose salary is greater than 10000.

4. **Provide modification or transaction support to user data :** means we can modify already stored data in database or delete data already stored in database
   Etc

b. Types of databases?

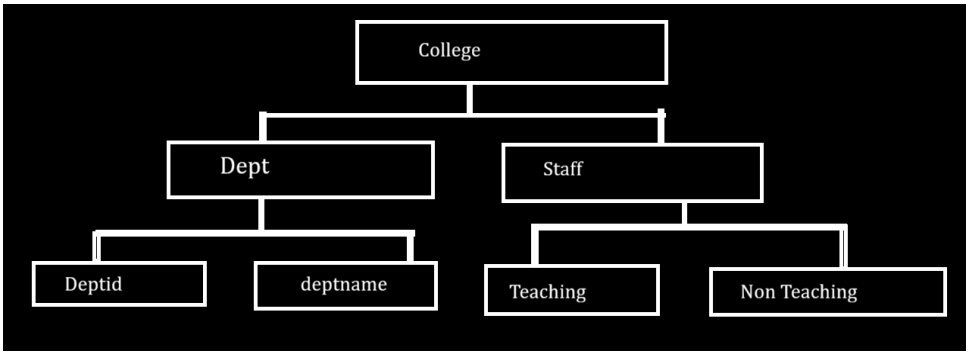
If we want to work with database we have some types of database

1. **Relational database:** Relational database means a database can store its data in tabular format or in table format called as relational database
   It contain row and columns or tuple and attribute
   In terms of relational database row indicate tuple and column indicate attributes



Example: MYSQL,Oracle etc

2. **Hierarchical database:** Hierarchical database means a database can store data in parent and child format or in parent and child relationship

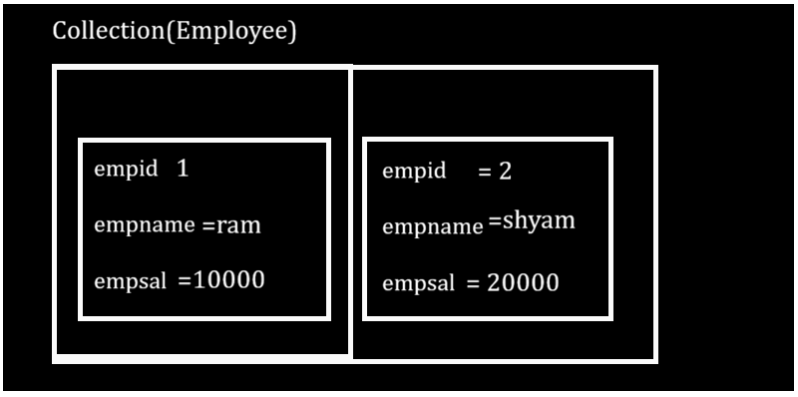


Example: Operating file and folder structure database

3. **Cloud database :** cloud database means database can store on remote server as well as we can use or update from client machine or user machine remotely called as cloud database and it can access by multiple user centrally Example: google spread sheet or google doc

4. **Distributed database:** distributed database means multiple database server can communicate with centralize data base or data center called as distributed database

5. **Object oriented database :** object oriented database means a data can store in object format and field format here collection of objects known as collection like as database table, object like as row and field or variable like as column



Etc

Now we want to discuss about the Relational database management system

Steps to work with Relational database management system

a. **Install any database tool like as MYSQL,Oracle etc**

   If we want to work with relational database management system we have to use tool or application which is responsible create environment to work with relational database like as MYSQL

   Q. What is MYSQL?

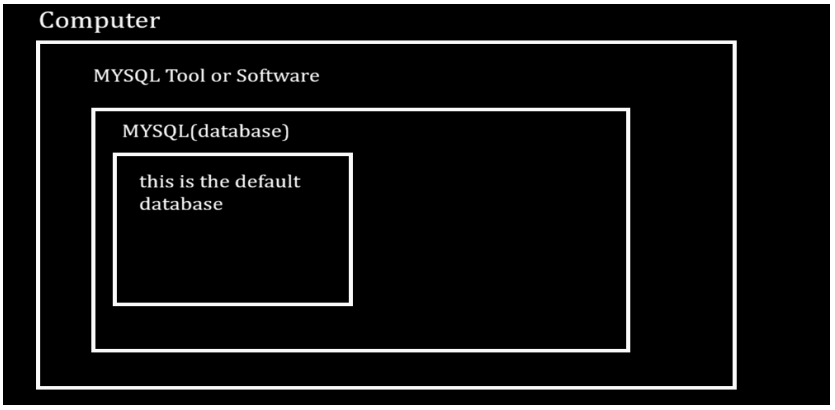
   MYSQL is application which provide environment to us to work with relational database management system

   Download & Install MYSQL using a given link

   https://dev.mysql.com/downloads/installer/

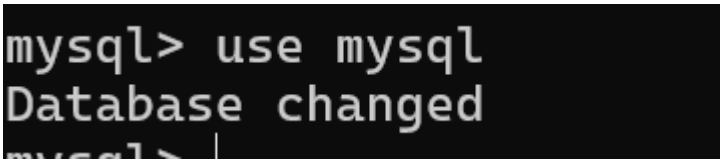
b. **Create your own database or use inbuilt database provided by tool**

If we think about Relational database we can database is a collection or tables
When we install mysql we get one default database known as mysql



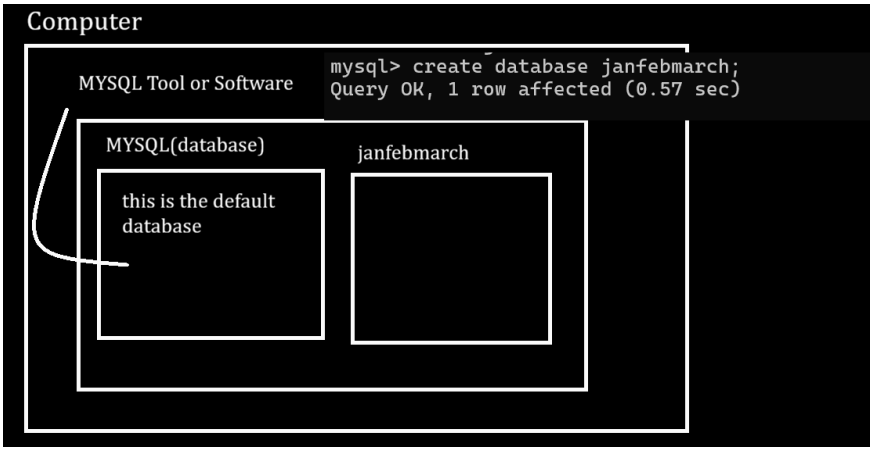
If we want to check the default database we have to use the following steps.

1. Open the mysql
2. Login to mysql
3. Use command : use mysql



Note: you can create your own database and if you want to create your own database we have to use following command

Syntax: create database databasename



If we think about above diagram we created successfully user database known as janfebmarch
Note: once we create database you can use it if want to use your own database we have syntax

Syntax: use databasename

c. **Work with a database like as create table, update table etc**
So if we want to work with database we have some following kinds of commands

1. **DDL :** DDL stands for data definition language basically this type of command help us to work with table structure, view structure etc
   Types of DDL command

   a. **Create**
   b. **Alter**
   c. **Drop**
   d. **Desc**
   e. **Truncate**

2. **DML:** DML stands for data manipulation language and it is used for store data in database table, delete data from table, update data in table etc means this type of command can work with table data not structure

   a. Insert
   b. Delete
   c. Update

3. **DQL :** Data Query language it is used for fetch data from table or select data from table
   a. **Select**
4. **TCL :** TCL stands for Transaction Control language
   a. **Commit**
   b. **Rollback**
   c. **Savepoint**
5. **DCL :** Data control language this type of command is used for provide privileges to user data

   a. **Grant**
   b. **Revoke**

Now we want to discuss about DDL command

1. **create:** create command is used for create table, create view, create procedure, create trigger , create index, create function as well as create database **etc**

   How to create database table using create command or statement

   Syntax: create table tablename(column_name datatype(size)....);
   Example: suppose consider we want to create employee table with field or column eid,name,esal

```
mysql>  create table employee(eid int(5),ename varchar(200),esal int(5));
Query OK, 0 rows affected, 2 warnings (0.41 sec)
```

2. **Alter statement :** alter statement is used for modify the table structure like as add new column in table, rename column name, drop column, change column name and size etc
   If we want to work with alter statement we have some options

   a. **Add :** this option is used for add new column in database table

Syntax:
alter table tablename add column columnname datatype(size): add single column using this statement
or
alter table tablename add column (columnname datatype(size),columnname datatype(size)): add multiple column using this command

```
mysql> desc employee;
+-------+--------------+------+-----+---------+-------+
| Field | Type         | Null | Key | Default | Extra |
+-------+--------------+------+-----+---------+-------+
| eid   | int          | YES  |     | NULL    |       |
| ename | varchar(200) | YES  |     | NULL    |       |
| esal  | int          | YES  |     | NULL    |       |
+-------+--------------+------+-----+---------+-------+
3 rows in set (0.06 sec)
```

If we think about above table structure we have three column in database table eid,ename and esal but we want two more column in table like as email and contact

```
mysql> alter table employee add column (email varchar(100),contact int(10));
Query OK, 0 rows affected, 1 warning (0.41 sec)
Records: 0  Duplicates: 0  Warnings: 1

mysql> desc employee;
+---------+--------------+------+-----+---------+-------+
| Field   | Type         | Null | Key | Default | Extra |
+---------+--------------+------+-----+---------+-------+
| eid     | int          | YES  |     | NULL    |       |
| ename   | varchar(200) | YES  |     | NULL    |       |
| esal    | int          | YES  |     | NULL    |       |
| email   | varchar(100) | YES  |     | NULL    |       |
| contact | int          | YES  |     | NULL    |       |
+---------+--------------+------+-----+---------+-------+
5 rows in set (0.00 sec)
```

   b. **Modify :** modify option is used for change column size or column type

Syntax: alter table tablename modify column columnname(datatype size);
Example:

```
mysql> alter table employee modify column email varchar(50);
Query OK, 0 rows affected (0.03 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> commit;
Query OK, 0 rows affected (0.00 sec)

mysql> desc employee;
+---------+--------------+------+-----+---------+-------+
| Field   | Type         | Null | Key | Default | Extra |
+---------+--------------+------+-----+---------+-------+
| eid     | int          | YES  |     | NULL    |       |
| ename   | varchar(200) | YES  |     | NULL    |       |
| esal    | int          | YES  |     | NULL    |       |
| email   | varchar(50)  | YES  |     | NULL    |       |
| contact | varchar(20)  | YES  |     | NULL    |       |
+---------+--------------+------+-----+---------+-------+
5 rows in set (0.01 sec)
```

c. **Drop:** drop option is used for remove the column from a database table

```
mysql> desc employee;
+---------+--------------+------+-----+---------+-------+
| Field   | Type         | Null | Key | Default | Extra |
+---------+--------------+------+-----+---------+-------+
| eid     | int          | YES  |     | NULL    |       |
| ename   | varchar(200) | YES  |     | NULL    |       |
| esal    | int          | YES  |     | NULL    |       |
| email   | varchar(50)  | YES  |     | NULL    |       |
| contact | varchar(20)  | YES  |     | NULL    |       |
+---------+--------------+------+-----+---------+-------+
5 rows in set (0.01 sec)
```

Suppose consider we have table name as employee with column eid,ename,esal,email and contact and we want to remove the email column from a database table

```
mysql> alter table employee drop column email;
Query OK, 0 rows affected (0.13 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> commit;
Query OK, 0 rows affected (0.00 sec)

mysql> desc employee;
+---------+--------------+------+-----+---------+-------+
| Field   | Type         | Null | Key | Default | Extra |
+---------+--------------+------+-----+---------+-------+
| eid     | int          | YES  |     | NULL    |       |
| ename   | varchar(200) | YES  |     | NULL    |       |
| esal    | int          | YES  |     | NULL    |       |
| contact | varchar(20)  | YES  |     | NULL    |       |
+---------+--------------+------+-----+---------+-------+
4 rows in set (0.01 sec)
```

d. **Rename:** this option help us to rename column name

Syntax: alter table tablename rename column old column to new column;

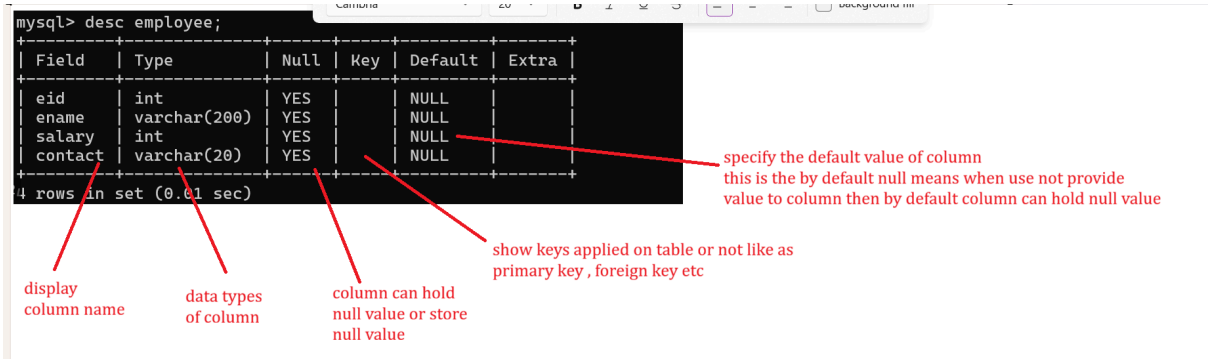
```
mysql> desc employee;
+---------+--------------+------+-----+---------+-------+
| Field   | Type         | Null | Key | Default | Extra |
+---------+--------------+------+-----+---------+-------+
| eid     | int          | YES  |     | NULL    |       |
| ename   | varchar(200) | YES  |     | NULL    |       |
| esal    | int          | YES  |     | NULL    |       |
| contact | varchar(20)  | YES  |     | NULL    |       |
+---------+--------------+------+-----+---------+-------+
4 rows in set (0.01 sec)
```

Suppose consider we have esal column in table and we want to change its name from esal to salary

```
mysql> desc employee;
+---------+--------------+------+-----+---------+-------+
| Field   | Type         | Null | Key | Default | Extra |
+---------+--------------+------+-----+---------+-------+
| eid     | int          | YES  |     | NULL    |       |
| ename   | varchar(200) | YES  |     | NULL    |       |
| salary  | int          | YES  |     | NULL    |       |
| contact | varchar(20)  | YES  |     | NULL    |       |
+---------+--------------+------+-----+---------+-------+
4 rows in set (0.00 sec)
```

3. Describe or desc: this command is used for describe the table structure means show the column name, its size , its type etc

Syntax: desc tablename
Example:



4. Drop: drop command is used for remove the column from a database table
Syntax: drop table tablename

```
mysql> drop table employee;
Query OK, 0 rows affected (0.03 sec)

mysql> desc employee;
ERROR 1146 (42S02): Table 'janfebmarch.employee' doesn't exist
```

5. Truncate: truncate statement is used for delete all records of table or data of table
Syntax: drop tablename;

DML Statement : DML command only works on table data
Types of DML statements

a. **Insert :** this is used for insert data in table or record in table

There are two ways to write insert command

---

1. **Wild card insert :** wild card insert means we can insert data in column in row called as wild card insert

Syntax: insert into tablename values(value1,value2….value…n);

Example of wild card insert

```
mysql> insert into employee values(1,'RAM',10000);
Query OK, 1 row affected (0.01 sec)

mysql> insert into employee values(2,'SHYAM',20000);
Query OK, 1 row affected (0.01 sec)
```

2. **Partial insert :** we can insert data in specific column of table called as partial insert

Syntax: insert into tablename (columnanem…)values(value…);

Note: in this case we have to specify column name before values

Suppose consider we have employee table with three field id,name and salary but we want to insert only id and name not salary

```
mysql> insert into employee (id,name)values(3,'XYZ');
Query OK, 1 row affected (0.01 sec)
```

b. **Delete :** delete statement is used for delete data from a database table

Types of delete

1. **Wild card delete :** wild card delete is used for delete all records from table or all rows from a database table

Syntax: delete from tablename

```
mysql> delete from employee;
```

2. **Partial delete:** partial delete means delete specific record from a database table using where condition

Syntax: delete from tablename where condition

```
mysql> delete from employee where id=3;
Query OK, 1 row affected (0.01 sec)
```

c. **Update :** if we want to modify table column data then we can use update statement

Types of update

1. **Wild card update :** update column value in all row

Syntax: update table tablename set column=value;

```
mysql> update employee set sal=40000;
Query OK, 2 rows affected (0.01 sec)
Rows matched: 2  Changed: 2  Warnings: 0

mysql> select *from employee;
+------+-------+-------+
| id   | name  | sal   |
+------+-------+-------+
|    1 | RAM   | 40000 |
|    2 | SHYAM | 40000 |
+------+-------+-------+
2 rows in set (0.00 sec)
```

2. **Partial update**

Syntax: update table tablename set column=value , column=value where condition: update specific record in table

```
mysql> update employee set sal=50000 where id=1;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> select  *from employee;
+------+-------+-------+--------+
| id   | name  | sal   | status |
+------+-------+-------+--------+
|    1 | RAM   | 50000 |      0 |
|    2 | SHYAM | 40000 |      0 |
+------+-------+-------+--------+
2 rows in set (0.00 sec)
```

DQL: this type of command is used for fetch data from a database table or select data from database table

Syntax: select *from tablename: fetch all row with all columns

```
mysql> select *from employee;
+------+-------+-------+--------+
| id   | name  | sal   | status |
+------+-------+-------+--------+
|    1 | RAM   | 50000 |      0 |
|    2 | SHYAM | 40000 |      0 |
+------+-------+-------+--------+
2 rows in set (0.00 sec)
```

Or

Syntax: select column1,column2…. From tablename : fetch all row or records with a specific column

```
mysql> select name,sal from employee;
+-------+-------+
| name  | sal   |
+-------+-------+
| RAM   | 50000 |
| SHYAM | 40000 |
+-------+-------+
2 rows in set (0.00 sec)
```

Or

Syntax: select * from tablename where condition: fetch row with specific condition with all columns

```
mysql> select *from employee where id=1;
+------+------+-------+--------+
| id   | name | sal   | status |
+------+------+-------+--------+
|    1 | RAM  | 50000 |      0 |
+------+------+-------+--------+
1 row in set (0.00 sec)
```

```
mysql> select name,sal from employee where id=1;
+------+-------+
| name | sal   |
+------+-------+
| RAM  | 50000 |
+------+-------+
1 row in set (0.00 sec)
```

Logical operator in SQL

---

a. **And :** if all conditions true then condition is true otherwise false

Syntax: select *from tablename where condition1 and condition2;

Or

Select *from tablename where condition1 && condition2;

Example: fetch employee details whose id is 1 and name is RAM

```
mysql> select *from employee where id=1 and name='RAM'
    -> ;
+------+------+-------+--------+
| id   | name | sal   | status |
+------+------+-------+--------+
|    1 | RAM  | 50000 |      0 |
+------+------+-------+--------+
1 row in set (0.00 sec)
```

b. **Or :** if any one condition is true then condition is true otherwise f**alse**
Syntax: select *from tablename where condition1 or condition2;

Or

Syntax: Select *from tablename where condition1 || condition2;

Example: select employee record whose id is 2 or name is ram

```
mysql> select *from employee where id=2 or name='RAM'
    -> ;
+------+-------+-------+--------+
| id   | name  | sal   | status |
+------+-------+-------+--------+
|    1 | RAM   | 50000 |      0 |
|    2 | SHYAM | 40000 |      0 |
+------+-------+-------+--------+
2 rows in set (0.00 sec)

mysql> select *from employee where id=2 || name='RAM';
+------+-------+-------+--------+
| id   | name  | sal   | status |
+------+-------+-------+--------+
|    1 | RAM   | 50000 |      0 |
|    2 | SHYAM | 40000 |      0 |
+------+-------+-------+--------+
2 rows in set, 1 warning (0.00 sec)
```

c. **Not :** not operator is used for reverse the condition means if your condition is true then false and if false then true

Syntax: select *from tablename where not condition;

Or

select *from tablename where column_name!=condition;

Or

Select *from tablename where column_name<>condition

Example: fetch employee data whose id is not 1

```
mysql> select *from employee where not id=1;
+------+-------+-------+--------+
| id   | name  | sal   | status |
+------+-------+-------+--------+
|    2 | SHYAM | 40000 |      0 |
|    3 | ABC   | 10000 |      0 |
|    4 | XYZ   | 20000 |      0 |
+------+-------+-------+--------+
3 rows in set (0.00 sec)

mysql> select *from employee where id!=1;
+------+-------+-------+--------+
| id   | name  | sal   | status |
+------+-------+-------+--------+
|    2 | SHYAM | 40000 |      0 |
|    3 | ABC   | 10000 |      0 |
|    4 | XYZ   | 20000 |      0 |
+------+-------+-------+--------+
3 rows in set (0.00 sec)

mysql> select *from employee where id<>1;
+------+-------+-------+--------+
| id   | name  | sal   | status |
+------+-------+-------+--------+
|    2 | SHYAM | 40000 |      0 |
|    3 | ABC   | 10000 |      0 |
|    4 | XYZ   | 20000 |      0 |
+------+-------+-------+--------+
3 rows in set (0.00 sec)
```

IN and between operator

IN: IN operator is used for avoid writing multiple OR conditions as well as IN operator is used as conjunction in SubQuery
When we use or operator with same column in every condition then IN operator is recommended

Syntax: select *from tablename where column_name IN(value1,value2,value…n)

Example: WAP to fetch employee details whose salary is 10000,20000,50000

```
mysql> select *from employee where sal=10000 or sal=20000 or sal=50000;
+------+------+-------+--------+
| id   | name | sal   | status |
+------+------+-------+--------+
|    1 | RAM  | 50000 |      0 |
|    3 | ABC  | 10000 |      0 |
|    4 | XYZ  | 20000 |      0 |
+------+------+-------+--------+
3 rows in set (0.00 sec)

mysql> select *from employee where sal IN(10000,20000,50000);
+------+------+-------+--------+
| id   | name | sal   | status |
+------+------+-------+--------+
|    1 | RAM  | 50000 |      0 |
|    3 | ABC  | 10000 |      0 |
|    4 | XYZ  | 20000 |      0 |
+------+------+-------+--------+
3 rows in set (0.00 sec)
```

Between operator: the goal of between operator is execute the range query means between operator is same as >= and <= condition

Syntax: select *from tablename where column between start range value and end range value

Example: WA SQL query fetch employee data whose salary is between 20000 to 50000

```
mysql> select *from employee where sal>=20000 and sal<=50000;
+------+-------+-------+--------+
| id   | name  | sal   | status |
+------+-------+-------+--------+
|    1 | RAM   | 50000 |      0 |
|    2 | SHYAM | 40000 |      0 |
|    4 | XYZ   | 20000 |      0 |
+------+-------+-------+--------+
3 rows in set (0.00 sec)

mysql> select *from employee where sal between 20000 and 50000;
+------+-------+-------+--------+
| id   | name  | sal   | status |
+------+-------+-------+--------+
|    1 | RAM   | 50000 |      0 |
|    2 | SHYAM | 40000 |      0 |
|    4 | XYZ   | 20000 |      0 |
+------+-------+-------+--------+
3 rows in set (0.00 sec)
```

Like operator : like operator is used for execute the pattern matching query or execute pattern in query
Basically like operator help us to work with a searching operation called as like operator

Syntax: select *from tablename where columnname like pattern

If we want to work with like operator we have following wild card operators

1. **% :** it represents zero or more character
2. **_ :** it represent single character

Example1: fetch employee whose name start with s

```
mysql> select *from employee where name like  's%';
+------+---------+-------+--------+
| id   | name    | sal   | status |
+------+---------+-------+--------+
|    2 | SHYAM   | 40000 |      0 |
|    6 | sanjay  | 40000 |      0 |
|    7 | sandeep | 50000 |      0 |
|    8 | santosh | 60000 |      0 |
+------+---------+-------+--------+
4 rows in set (0.00 sec)
```

Example: fetch employee data whose name ends with sh

```
mysql> select *from employee where name like  '%sh';
+------+---------+-------+--------+
| id   | name    | sal   | status |
+------+---------+-------+--------+
|    5 | dinesh  | 45000 |      0 |
|    8 | santosh | 60000 |      0 |
+------+---------+-------+--------+
2 rows in set (0.00 sec)
```

Example: Fetch employee data whose name start with s and ends with sh

```
mysql> select *from employee where name like  's%sh';
+------+---------+-------+--------+
| id   | name    | sal   | status |
+------+---------+-------+--------+
|    8 | santosh | 60000 |      0 |
+------+---------+-------+--------+
1 row in set (0.00 sec)
```

Example: WA SQL Query to fetch employee data whose name contain only three letters

```
mysql> select *from employee where name like '___';
+------+------+-------+--------+
| id   | name | sal   | status |
+------+------+-------+--------+
|    1 | RAM  | 50000 |      0 |
|    3 | ABC  | 10000 |      0 |
|    4 | XYZ  | 20000 |      0 |
+------+------+-------+--------+
3 rows in set (0.00 sec)
```

Example: WA SQL Query to fetch employee data whose name contain at least three letters

```
mysql> select *from employee where name like '___%';
+------+---------+-------+--------+
| id   | name    | sal   | status |
+------+---------+-------+--------+
|    1 | RAM     | 50000 |      0 |
|    2 | SHYAM   | 40000 |      0 |
|    3 | ABC     | 10000 |      0 |
|    4 | XYZ     | 20000 |      0 |
|    5 | dinesh  | 45000 |      0 |
|    6 | sanjay  | 40000 |      0 |
|    7 | sandeep | 50000 |      0 |
|    8 | santosh | 60000 |      0 |
+------+---------+-------+--------+
8 rows in set (0.00 sec)
```

Aggregate function

Aggregate function is used for generate collective result or perform operation on column and generate single value as result

Types of aggregate function

a. **Sum :** this function is used for calculate sum of all values in column and generate the single value as result called as aggregate function
   Example: suppose consider we have employee table and we want to calculate sum of all employee salary
   Syntax: select sum(column_name) from tablename;

```
mysql> select sum(sal) from employee;
+-----------+
| sum(sal) |
+-----------+
|   363888 |
+-----------+
1 row in set (0.02 sec)
```

b. **Count :** count function is used for count the records of table
   There are two types of count syntax
   Syntax1: select count(column_name) from tablename: if we pass column name in count function then function can count only non null values.

```
mysql> select *from employee;
+------+---------+-------+--------+
| id   | name    | sal   | status |
+------+---------+-------+--------+
|    1 | RAM     | 50000 |      0 |
|    2 | SHYAM   | 40000 |      0 |
|    3 | ABC     | 10000 |      0 |
|    4 | XYZ     | 20000 |      0 |
|    5 | dinesh  | 45000 |      0 |
|    6 | sanjay  | 40000 |      0 |
|    7 | sandeep | 50000 |      0 |
|    8 | santosh | 60000 |      0 |
|    9 | a       |  4444 |      0 |
|    9 | b       | 44444 |      0 |
|   10 | NULL    | 44444 |      0 |
|   11 | NULL    | 54444 |      0 |
+------+---------+-------+--------+
12 rows in set (0.00 sec)
```

```
mysql> select count(name) from employee;
+-------------+
| count(name) |
+-------------+
|          10 |
+-------------+
1 row in set (0.00 sec)
```

Note: if we think about above select query with count function we have 12 records in employee table but query return 10 count because we have two employee records in which employee name is null

   Syntax2: select column(*) from tablename: if we pass * In the count function then we can count null values as well as non null values.

```
mysql> select *from employee;
+------+---------+-------+--------+
| id   | name    | sal   | status |
+------+---------+-------+--------+
|    1 | RAM     | 50000 |      0 |
|    2 | SHYAM   | 40000 |      0 |
|    3 | ABC     | 10000 |      0 |
|    4 | XYZ     | 20000 |      0 |
|    5 | dinesh  | 45000 |      0 |
|    6 | sanjay  | 40000 |      0 |
|    7 | sandeep | 50000 |      0 |
|    8 | santosh | 60000 |      0 |
|    9 | a       |  4444 |      0 |
|    9 | b       | 44444 |      0 |
|   10 | NULL    | 44444 |      0 |
|   11 | NULL    | 54444 |      0 |
+------+---------+-------+--------+
12 rows in set (0.00 sec)
```

```
mysql> select count(*) from employee;
+----------+
| count(*) |
+----------+
|       12 |
+----------+
1 row in set (0.05 sec)
```

Note: this query count null as well as non null values.

c. **Avg: avg function is used for calculate average value of column**

Syntax: select avg(column_name) from tablename;

```
mysql> select avg(sal) from employee;
+------------+
| avg(sal)   |
+------------+
| 36388.8000 |
+------------+
1 row in set (0.00 sec)
```

The logic of above code is

```
mysql> select sum(sal)/count(sal) from employee;
+---------------------+
| sum(sal)/count(sal) |
+---------------------+
|          36388.8000 |
+---------------------+
1 row in set (0.00 sec)
```

d. **Max :** max function is used to find the max value from the table.

```
mysql> select max(sal) from employee;
+----------+
| max(sal) |
+----------+
|    60000 |
+----------+
1 row in set (0.01 sec)
```

e. **Min : this function is used for find the minimum value from employee**

Syntax: select min(column_name) from tablename;

Example:

```
mysql> select min(sal) from employee;
+-----------+
| min(sal)  |
+-----------+
|      4444 |
+-----------+
1 row in set (0.00 sec)
```

Group by clause

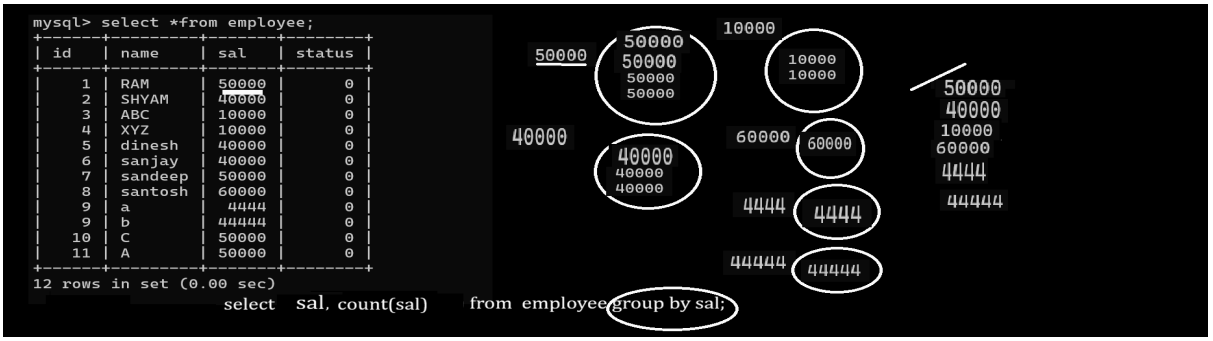
Group by clause is used for group the column values and generate single value as result

Syntax: select column_name from tablename group by columname

If we want to work with group by clause we have some important rules

a. Group by clause always work with aggregate function
b. You can use column name in select query whose name used in group by clause

Example: WA SQL Query find the employee count whose salary is same



```
mysql> select sal,count(sal) from employee group by sal;
+-------+------------+
| sal   | count(sal) |
+-------+------------+
| 50000 |          4 |
| 40000 |          3 |
| 10000 |          2 |
| 60000 |          1 |
|  4444 |          1 |
| 44444 |          1 |
+-------+------------+
6 rows in set (0.01 sec)
```

Having clause

Having clause work with group by clause and goal of having clause is check the condition with group by clause

Important points related with having clause

a. Always work with group by clause
b. You can use aggregate function in having clause for check the condition
c. Having clause always use after group by clause

Example: Find the duplicated salary from table

```
mysql> select sal, count(sal) from employee group by sal having count(sal)>1;
+-------+------------+
| sal   | count(sal) |
+-------+------------+
| 50000 |          4 |
| 40000 |          3 |
| 10000 |          2 |
+-------+------------+
3 rows in set (0.01 sec)
```

Order by clause: this clause is used for arrange your data in ascending order or in descending order

Syntax: select *from tablename order by column_name asc | desc

Example:

```
mysql> select *from employee order by sal desc;
+------+---------+-------+--------+
| id   | name    | sal   | status |
+------+---------+-------+--------+
|    8 | santosh | 60000 |      0 |
|    1 | RAM     | 50000 |      0 |
|    7 | sandeep | 50000 |      0 |
|   10 | C       | 50000 |      0 |
|   11 | A       | 50000 |      0 |
|    9 | b       | 44444 |      0 |
|    2 | SHYAM   | 40000 |      0 |
|    5 | dinesh  | 40000 |      0 |
|    6 | sanjay  | 40000 |      0 |
|    3 | ABC     | 10000 |      0 |
|    4 | XYZ     | 10000 |      0 |
|    9 | a       |  4444 |      0 |
+------+---------+-------+--------+
12 rows in set (0.00 sec)
```

Constraints in SQL

Constrains means apply the restrictions on table column at the time of table creation called as constraints

Types of constraints

---

a. **Not null :** not null constraints is used for avoid to store null value in column called as not null constraints
When we apply not null constraints on the table column then the column cannot accept null values.

Syntax: create table tablename(column_name datatype(size) not null….)
mysql> create table employee(id int(5),name varchar(200) not null, email varchar(200),contact int(10));
Query OK, 0 rows affected, 2 warnings (0.20 sec)

```
mysql> insert into employee values(1,null,'ram@gmail.com',123456);
ERROR 1048 (23000): Column 'name' cannot be null
```

Note: above query generate error to us because we try to store null value in name column but we apply not null constraints on name column
The meaning of above statement is we must be pass value to name column

```
mysql> insert into employee values(1,'ram','ram@gmail.com',123456);
Query OK, 1 row affected (0.01 sec)
```

b. **Unique:** when we use the unique constraints on the column then the column does not accept duplicated values. You must have to store unique value in that column

Syntax: create table tablename(column_name datatype(size) unique)
Or
Syntax: create table tablename(column_name datatype(size) not null unique)

Example: create table name as employee with field id,name,email ,contact and salary but the employee email and salary should not be duplicated

mysql> create table employee(id int(5) , name varchar(200) not null, email varchar(50) unique,contact varchar(10) unique,sal int(5));
Query OK, 0 rows affected, 2 warnings (0.14 sec)

```
mysql> insert into employee values(1,'ram','ram@gmail.com','12456789',10000);
Query OK, 1 row affected (0.01 sec)

mysql> insert into employee values(2,'ram','ram@gmail.com','12456789',10000);
ERROR 1062 (23000): Duplicate entry 'ram@gmail.com' for key 'employee.email'
mysql> insert into employee values(2,'ram','rama@gmail.com','12456789',10000);
ERROR 1062 (23000): Duplicate entry '12456789' for key 'employee.contact'
mysql>
```

c. **Primary :** primary key constraints is combination of unique key and not null means we cannot store null value in primary key as well as cannot store duplicated value in primary key column

Syntax: create table tablename(column_name datatype(size) primary key,column_name datatype(size)....);

```
mysql> create table student(sid int(5) primary key,name varchar(200) not null,email
varchar(200) not null unique ,contact varchar(10) not null unique);
Query OK, 0 rows affected, 1 warning (0.14 sec)

mysql> insert into student values(1,'a','a@gmail.com','1234655');
Query OK, 1 row affected (0.02 sec)

mysql> insert into student values(1,'a','a@gmail.com','1234655');
ERROR 1062 (23000): Duplicate entry '1' for key 'student.PRIMARY'
mysql>
```

Q. What is the difference between primary and unique?

---

a. Primary key is by default not null unique but unique use by default null value
b. Cannot create more one primary key column in single table but there may be more than one unique column in database table
   Or unique key in database table
c. Primary key use by default cluster indexing or primary index and unique us by default non cluster index i.e secondary index

d. **Foreign :** foreign key constraints is used for provide relationship between two tables means create parent and child relationship between two tables
Normally primary key column table is parent table and foreign key column is a child table

How to create foreign key column

Syntax: Create table tablename(column_name datatype(size) , foreign key (columnname) references parenttable(primary key column_name));

Example:

```
mysql> create table course(cid int(5) primary key, cname varchar(200) not null
unique);
mysql> create table student(sid int(5) primary key, sname varchar(200) not null, cid
int(5),foreign key(cid) references course(cid));
```

Example with source code

```
mysql> insert into course values(1,'JAVA');
Query OK, 1 row affected (0.01 sec)

mysql> insert into course values(2,'PYTHON');
Query OK, 1 row affected (0.01 sec)

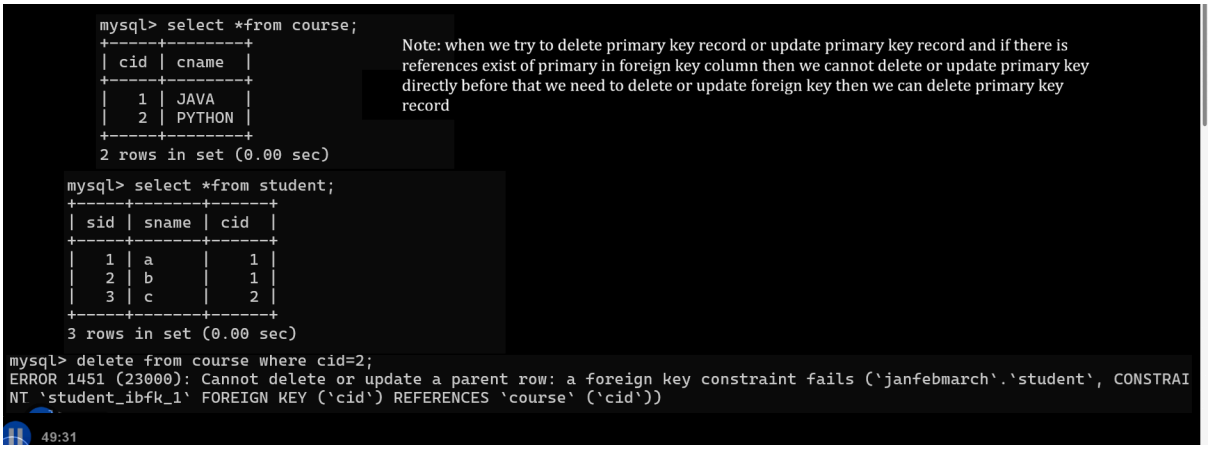
mysql> insert into student values(1,'a',1);
Query OK, 1 row affected (0.02 sec)
```

mysql> insert into student values(2,'b',1);
Query OK, 1 row affected (0.01 sec)

mysql> insert into student values(3,'c',2);
Query OK, 1 row affected (0.01 sec)

```
mysql> select *from course;
+-----+--------+
| cid | cname  |
+-----+--------+
|   1 | JAVA   |
|   2 | PYTHON |
+-----+--------+
2 rows in set (0.00 sec)

mysql> select *from student;
+-----+-------+------+
| sid | sname | cid  |
+-----+-------+------+
|   1 | a     |    1 |
|   2 | b     |    1 |
|   3 | c     |    2 |
+-----+-------+------+
3 rows in set (0.00 sec)
```



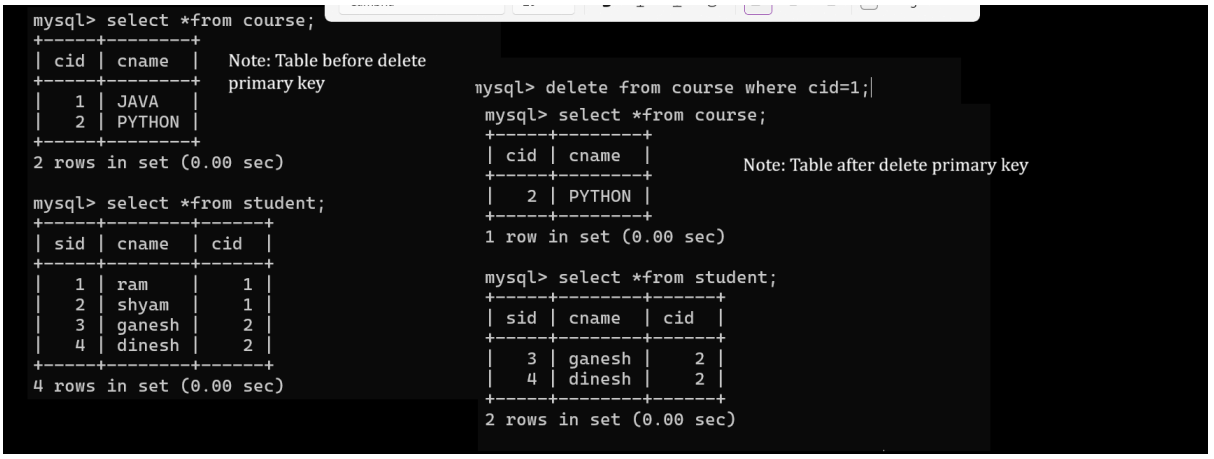
Note: if we want to solve this problem database provide some new constraints to us which we can use with foreign key and constraints name like as

e. **On delete cascade :** on delete cascade means when we delete parent record then child record should delete automatically

Syntax: create table childtablename(column_name datatype(size),foreign key(column_name) references parenttable(column_name) on delete cascade)

Example:
mysql> create table course(cid int(5) primary key, cname varchar(200) not null unique);

mysql> create table student(sid int(5) primary key , cname varchar(200),cid int(5) ,foreign key(cid) references course(cid) on delete cascade);

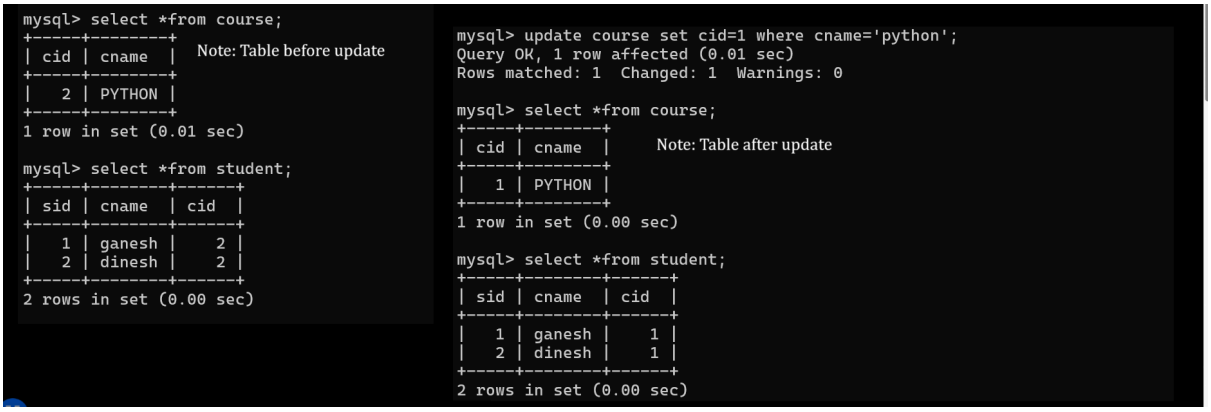


f. **On update cascade :** on update cascade these constraints can be used with foreign keys and when we update the primary key then automatically foreign keys get updated.

Syntax: create table childtablename(column_name datatype(size),foreign key(column_name) references parenttable(column_name) on delete cascade)

Example:
mysql> create table course(cid int(5) primary key, cname varchar(200) not null unique);

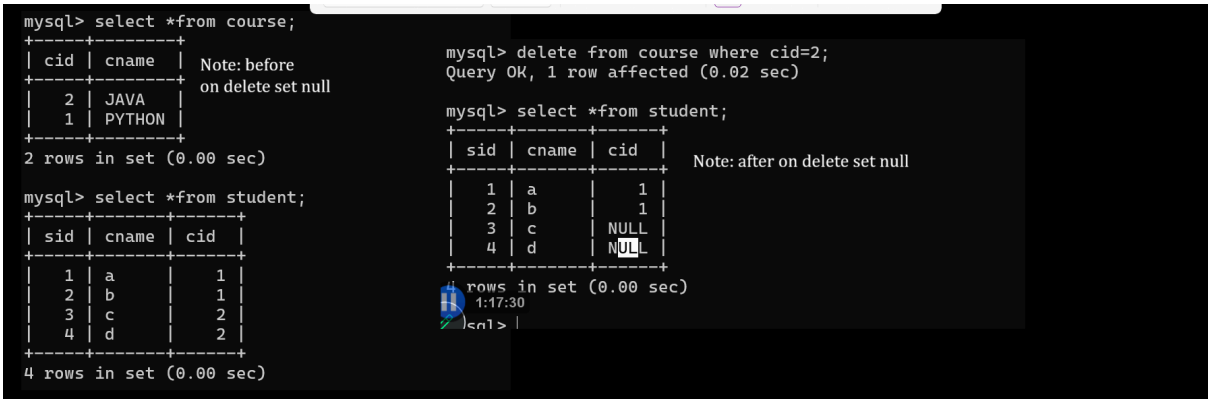
mysql> create table student(sid int(5) primary key , cname varchar(200),cid int(5) ,foreign key(cid) references course(cid) on update cascade);



g. **On delete set null:** this constraints not delete child record or foreign key record when we delete primary key just set child record as null
Syntax: create table childtablename(column_name datatype(size),foreign key(column_name) references parenttable(column_name) on delete set null)

Example:
mysql> create table course(cid int(5) primary key, cname varchar(200) not null unique);

mysql> create table student(sid int(5) primary key , cname varchar(200),cid int(5) ,foreign key(cid) references course(cid) on delete set null);



Note: you can use on delete cascade and on update cascade at same time with foreign key as well as on update cascade and on delete set null at same time
But cannot use on delete cascade and on delete set null at same time

h. **Auto_increment :** this constraints normally we use with primary key or unique and it help us to increase primary key value automatically

Syntax: create table tablename(column_name datatype(size) primary key auto_increment , column_name datatype(size).....);

Example: mysql> create table player(pid int(5) primary key auto_increment,name varchar(200),run int(5));

```
mysql> insert into player values('0','ram',10000);
Query OK, 1 row affected (0.09 sec)

mysql> insert into player values('0','shyam',20000);
Query OK, 1 row affected (0.01 sec)

mysql> insert into player values('0','ganesh',30000);
Query OK, 1 row affected (0.01 sec)
```

```
mysql> select *from player;
+-----+--------+-------+
| pid | name   | run   |
+-----+--------+-------+
|   1 | ram    | 10000 |
|   2 | shyam  | 20000 |
|   3 | ganesh | 30000 |
+-----+--------+-------+
3 rows in set (0.02 sec)
```

i. **Check :** check constraints is used for apply specific condition on column at the time of table creation and if condition satisfy then table can accept data otherwise not

Syntax: create table tablename(columname datatype(size) , check condition……)
Example: we want to create table name as voter with id,name and age and if voter age is greater than 18 then voter data should stored in database table otherwise not

```
mysql> create table voter(vid int(5) primary key auto_increment, name varchar(200),age int(5), check (age>18));
Query OK, 0 rows affected, 2 warnings (0.11 sec)
```

```
mysql> create table voter(vid int(5) primary key auto_increment, name varchar(200),age int(5), check (age>18));
Query OK, 0 rows affected, 2 warnings (0.11 sec)

mysql> insert into voter values('0','ram',20);
Query OK, 1 row affected (0.02 sec)

mysql> insert into voter values('0','shyam',30);
Query OK, 1 row affected (0.01 sec)

mysql> insert into voter values('0','shyam',10);
ERROR 3819 (HY000): Check constraint 'voter_chk_1' is violated.
mysql>
```

j. **Default** : default constraints is used for store the default value in column means when not set default value then by default database use null as default value
But user can set own default value in column as per his requirement

Syntax: create table tablename(column_name datatype(size) default value)

Example: suppose consider we want to create table name as batch with field id,name, placement status

```
mysql> create table batch(bid int(5) primary key auto_increment,batchname varchar(200), status int(5) default 0);
Query OK, 0 rows affected, 2 warnings (0.13 sec)

mysql> insert into batch (bid,batchname)values('0','JAN');
Query OK, 1 row affected (0.03 sec)

mysql> select *from batch;
+-----+-----------+--------+
| bid | batchname | status |
+-----+-----------+--------+
|   1 | JAN       |      0 |
+-----+-----------+--------+
1 row in set (0.01 sec)
```

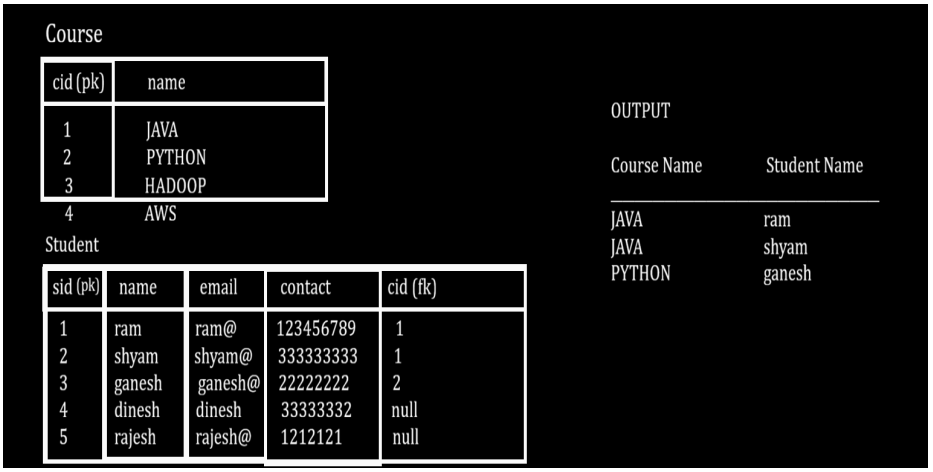
Joins in SQL

Joins is used for fetch data from multiple tables those are interconnected with each other using primary and foreign key called as joins

Types of joins

---

1. **Inner join :** inner join means fetch common data from left hand side table and right hand side table called as inner join



2. **Left join :** left join means fetch all records from left hand side table and common record from right hand side table

Course

cid (pk)	name
1	JAVA
2	PYTHON
3	HADOOP
4	AWS

Student

sid (pk)	name	email	contact	cid (fk)
1	ram	ram@	123456789	1
2	shyam	shyam@	333333333	1
3	ganesh	ganesh@	22222222	2
4	dinesh	dinesh	33333332	null
5	rajesh	rajesh@	1212121	null

OUTPUT

Course Name	Student Name
JAVA	ram
JAVA	shyam
PYTHON	ganesh
HADOOP	NULL
AWS	NULL

3. **Right join :** right join means fetch common records from both tables, meaning right hand side table and left hand side table as well as all records from right hand side table.

Course

cid (pk)	name
1	JAVA
2	PYTHON
3	HADOOP
4	AWS

Student

sid (pk)	name	email	contact	cid (fk)
1	ram	ram@	123456789	1
2	shyam	shyam@	333333333	1
3	ganesh	ganesh@	22222222	2
4	dinesh	dinesh	33333332	null
5	rajesh	rajesh@	1212121	null

OUTPUT

Course Name	Student Name
JAVA	ram
JAVA	shyam
PYTHON	ganesh
NULL	dinesh
NULL	rajesh

4. **Outer join :** Outer join means fetch all records from both side table means from left hand side table and right hand side table

Course

cid (pk)	name
1	JAVA
2	PYTHON
3	HADOOP
4	AWS

Student

sid (pk)	name	email	contact	cid (fk)
1	ram	ram@	123456789	1
2	shyam	shyam@	333333333	1
3	ganesh	ganesh@	22222222	2
4	dinesh	dinesh	33333332	null
5	rajesh	rajesh@	1212121	null

OUTPUT

Course Name	Student Name
JAVA	ram
JAVA	shyam
PYTHON	ganesh
NULL	dinesh
NULL	rajesh
HADOOP	NULL
AWS	NULL

5. **Cross join :** cross join create cartesian product between two tables record
6. **Self join :** self join is used for apply join on same table

How to apply join practically on table

Syntax:

Select ltref.col,rtref.col from lefttablename ltref join type righttablename rtf
on lft.column=rtf.column

```
mysql> select c.cname  as 'Course name' , s.cname as 'Student name' from course c inner join student s on c.cid=s.cid;
+-------------+--------------+
| Course name | Student name |
+-------------+--------------+
| JAVA        | ram          |
| JAVA        | shyam        |
| PYTHON      | ganesh       |
+-------------+--------------+
3 rows in set (0.02 sec)
```

Example of left join

```
mysql> select c.cname  as 'Course name' , s.cname as 'Student name' from course c left join student s on c.cid=s.cid;
+-------------+--------------+
| Course name | Student name |
+-------------+--------------+
| AWS         | NULL         |
| HADOOP      | NULL         |
| JAVA        | shyam        |
| JAVA        | ram          |
| PYTHON      | ganesh       |
+-------------+--------------+
5 rows in set (0.02 sec)
```

Example of right join

```
mysql> select c.cname  as 'Course name' , s.cname as 'Student name' from course c right join student s on c.cid=s.cid;
+-------------+--------------+
| Course name | Student name |
+-------------+--------------+
| JAVA        | ram          |
| JAVA        | shyam        |
| PYTHON      | ganesh       |
| NULL        | dinesh       |
| NULL        | rajesh       |
+-------------+--------------+
5 rows in set (0.00 sec)
```

Example of outer join

mysql> select c.cname as 'Course name' , s.cname as 'Student name' from course c left join student s on c.cid=s.cid union select c.cname as 'Course name',s.cname as 'Student name' from course c right join student s on c.cid=s.cid;

```
+-------------+--------------+
| Course name | Student name |
+-------------+--------------+
| AWS         | NULL         |
| HADOOP      | NULL         |
| JAVA        | shyam        |
| JAVA        | ram          |
| PYTHON      | ganesh       |
| NULL        | dinesh       |
| NULL        | rajesh       |
+-------------+--------------+
7 rows in set (0.02 sec)
```

Practice on joins

Consider we have following tables

1. mysql> create table courses(course_id int(5) primary key auto_increment, course_name varchar(100),duration_month int(5),fees int(5),trainer_name varchar(200));
2. mysql> create table students(student_id int(5) primary key auto_increment,student_name varchar(200),email varchar(200),city varchar(200),mars int(5),course_id int(5),foreign key(course_id) references courses(course_id));

Insert data in courses table

```
INSERT INTO courses (course_name, duration_month, fees, trainer_name)
   -> VALUES ('Java Full Stack', 6, 60000, 'Rohit'),
   ->        ('Python Full Stack', 5, 50000, 'Priya'),
   ->     ('Data Science', 8, 75000, 'Amit'),
   ->     ('React Development', 3, 30000, 'Sneha'),
   ->     ('Spring Boot', 4, 40000, 'Rohit');
Query OK, 5 rows affected (0.01 sec)
Records: 5  Duplicates: 0  Warnings: 0
```

Insert data in students table

```
INSERT INTO students (student_name, email, city, course_id, marks)
   ->      VALUES ('Aditya', 'aditya@gmail.com', 'Pune', 1, 85),
   ->             ('Rahul', 'rahul@gmail.com', 'Mumbai', 1, 72),
   ->             ('Sneha', 'sneha@gmail.com', 'Pune', 2, 90),
   ->             ('Priya', 'priya@gmail.com', 'Nashik', 3, 88),
   ->             ('Amit', 'amit@gmail.com', 'Pune', 1, 65),
   ->             ('Neha', 'neha@gmail.com', 'Mumbai', 4, 78),
   ->             ('Rohan', 'rohan@gmail.com', 'Pune', 5, 82),
   ->             ('Kiran', 'kiran@gmail.com', 'Satara', NULL, 70);
Query OK, 8 rows affected (0.01 sec)
Records: 8  Duplicates: 0  Warnings: 0
```


Example: Find the student with their course name

mysql> select c.course_name,s.student_name from courses c inner join students s on c.course_id=s.course_id;

```
+-------------------+--------------+
| course_name       | student_name |
+-------------------+--------------+
| Java Full Stack   | Aditya       |
| Java Full Stack   | Rahul        |
| Java Full Stack   | Amit         |
| Python Full Stack | Sneha        |
| Data Science      | Priya        |
| React Development | Neha         |
| Spring Boot       | Rohan        |
+-------------------+--------------+
7 rows in set (0.00 sec)
```

Example: WA SQL Query to find the student who did not enroll in any course?
Solution:
select s.student_name,count(s.course_id) from courses c right join students s on c.course_id=s.course_id group by student_id having count(s.course_id)=0

```
+--------------+--------------------+
| student_name | count(s.course_id) |
+--------------+--------------------+
| Kiran        |                  0 |
+--------------+--------------------+
1 row in set (0.00 sec)
```

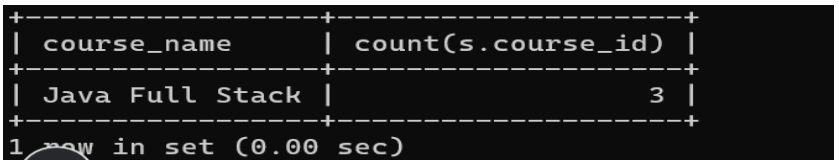
Example: Find the student list enrolled in Java Full Stack?
mysql> select c.course_name,s.student_name from courses c inner join students s on c.course_id=s.course_id where c.course_name='Java Full Stack';

```
+-----------------+--------------+
| course_name     | student_name |
+-----------------+--------------+
| Java Full Stack | Aditya       |
| Java Full Stack | Rahul        |
| Java Full Stack | Amit         |
+-----------------+--------------+
3 rows in set (0.00 sec)
```

Example: find the student whose name start with r and score more than 80%
mysql> select c.course_name,s.student_name,s.marks from courses c inner join students s on c.course_id=s.course_id where s.marks>80 and s.student_name like 'R%';

```
+-------------+--------------+-------+
| course_name | student_name | marks |
+-------------+--------------+-------+
| Spring Boot | Rohan        |    82 |
+-------------+--------------+-------+
1 row in set (0.00 sec)
```

Example: WA SQL query to find the course name in which more than one students enrolled
mysql> select c.course_name,count(s.course_id) from courses c inner join students s on c.course_id=s.course_id group by c.course_id having count(s.course_id)>1;

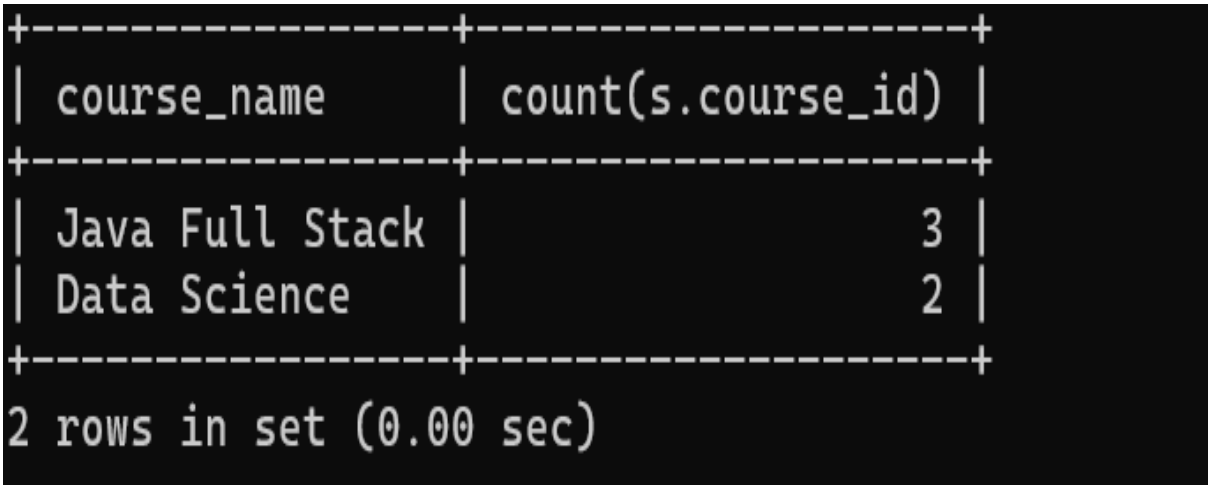


```
+-----------------+---------------------+
| course_name     | count(s.course_id) |
+-----------------+---------------------+
| Java Full Stack |                   3 |
+-----------------+---------------------+
1 row in set (0.00 sec)
```

Example: find the course in which more than one students exist and course name start with J or D and show course list in descending according to student count

Example with source code

mysql> select c.course_name,count(s.course_id) from courses c inner join students s on c.course_id=s.course_id where c.course_name like 'J%' or c.course_name like 'D%' group by c.course_id having count(s.course_id)>1 order by count(s.course_id) desc;



```
+-----------------+---------------------+
| course_name     | count(s.course_id) |
+-----------------+---------------------+
| Java Full Stack |                   3 |
| Data Science    |                   2 |
+-----------------+---------------------+
2 rows in set (0.00 sec)
```

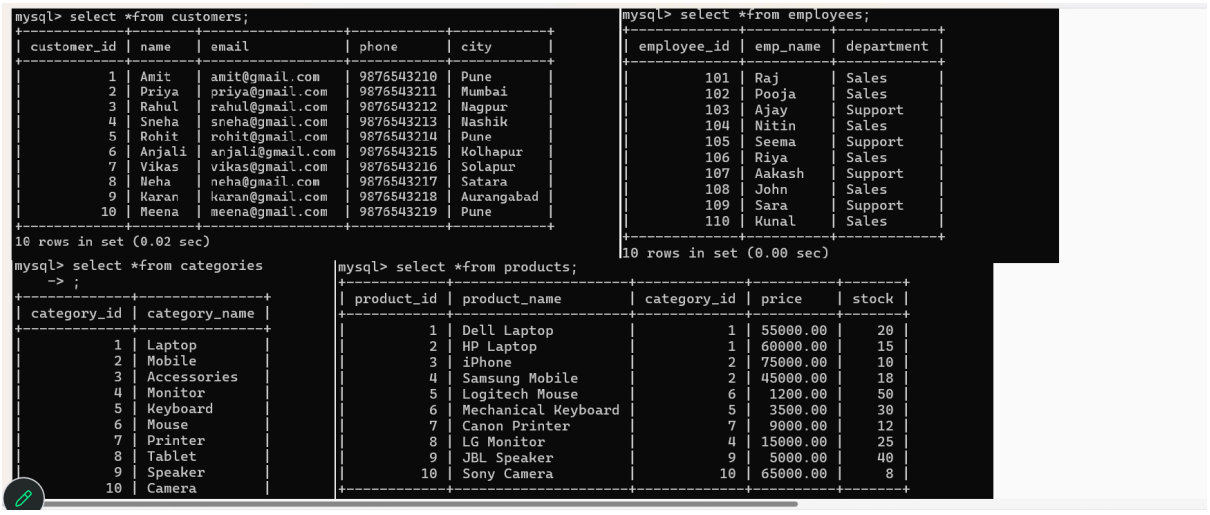
How to apply join on more than two tables

Rule - number of table - 1 join keyword in query

mysql> select c.name as 'Course name', s.name as 'Student name' from course c inner join csjoin cs on c.cid=cs.cid inner join student s on cs.sid=s.sid;

Example: find course wise student with their batches
mysql> select c.name as 'Course name', s.name as 'Student name',b.batchname as 'Batch Name' from course c inner join csjoin cs on c.cid=cs.cid inner join student s on s.sid=cs.sid inner join batch b on b.batchid=cs.batchid;

Example: Consider we are working on Order management database and you have following table structures



```
mysql> select *from customers;
+-------------+--------+-------------------+------------+------------+
| customer_id | name   | email             | phone      | city       |
+-------------+--------+-------------------+------------+------------+
|           1 | Amit   | amit@gmail.com    | 9876543210 | Pune       |
|           2 | Priya  | priya@gmail.com   | 9876543211 | Mumbai     |
|           3 | Rahul  | rahul@gmail.com   | 9876543212 | Nagpur     |
|           4 | Sneha  | sneha@gmail.com   | 9876543213 | Nashik     |
|           5 | Rohit  | rohit@gmail.com   | 9876543214 | Pune       |
|           6 | Anjali | anjali@gmail.com  | 9876543215 | Kolhapur   |
|           7 | Vikas  | vikas@gmail.com   | 9876543216 | Solapur    |
|           8 | Neha   | neha@gmail.com    | 9876543217 | Satara     |
|           9 | Karan  | karan@gmail.com   | 9876543218 | Aurangabad |
|          10 | Meena  | meena@gmail.com   | 9876543219 | Pune       |
+-------------+--------+-------------------+------------+------------+
10 rows in set (0.02 sec)

mysql> select *from employees;
+-------------+----------+------------+
| employee_id | emp_name | department |
+-------------+----------+------------+
|         101 | Raj      | Sales      |
|         102 | Pooja    | Sales      |
|         103 | Ajay     | Support    |
|         104 | Nitin    | Sales      |
|         105 | Seema    | Support    |
|         106 | Riya     | Sales      |
|         107 | Aakash   | Support    |
|         108 | John     | Sales      |
|         109 | Sara     | Support    |
|         110 | Kunal    | Sales      |
+-------------+----------+------------+
10 rows in set (0.00 sec)

mysql> select *from categories
    -> ;
+-------------+---------------+
| category_id | category_name |
+-------------+---------------+
|           1 | Laptop        |
|           2 | Mobile        |
|           3 | Accessories   |
|           4 | Monitor       |
|           5 | Keyboard      |
|           6 | Mouse         |
|           7 | Printer       |
|           8 | Tablet        |
|           9 | Speaker       |
|          10 | Camera        |
+-------------+---------------+

mysql> select *from products;
+------------+--------------------+-------------+----------+-------+
| product_id | product_name       | category_id | price    | stock |
+------------+--------------------+-------------+----------+-------+
|          1 | Dell Laptop        |           1 | 55000.00 |    20 |
|          2 | HP Laptop          |           1 | 60000.00 |    15 |
|          3 | iPhone             |           2 | 75000.00 |    10 |
|          4 | Samsung Mobile     |           2 | 45000.00 |    18 |
|          5 | Logitech Mouse     |           6 |  1200.00 |    50 |
|          6 | Mechanical Keyboard |           5 |  3500.00 |    30 |
|          7 | Canon Printer      |           7 |  9000.00 |    12 |
|          8 | LG Monitor         |           4 | 15000.00 |    25 |
|          9 | JBL Speaker        |           9 |  5000.00 |    40 |
|         10 | Sony Camera        |          10 | 65000.00 |     8 |
+------------+--------------------+-------------+----------+-------+
```

Q1. WA SQL query to find the customer and order details?

mysql> select c.name,o.order_id,o.order_date from customers c inner join orders o on c.customer_id=o.customer_id;

Q2. fetch product and its category?

Q3. fetch customer detail, product details and purchased details?

mysql> select c.name,p.product_name,p.price,o.order_date,os.quantity,o.order_id,(p.price*os.quantity) as 'Total bill' from customers c inner join orders o on c.customer_id=o.customer_id inner join order_items os on o.order_id=os.order_id inner join products p on p.product_id=os.product_id;

name	product_name	price	order_date	quantity	order_id	Total bill
Amit	Dell Laptop	55000.00	2026-08-01	1	201	55000.00
Priya	iPhone	75000.00	2026-08-02	1	202	75000.00
Rahul	Logitech Mouse	1200.00	2026-08-03	2	203	2400.00
Sneha	LG Monitor	15000.00	2026-08-04	1	204	15000.00
Rohit	HP Laptop	60000.00	2026-08-05	1	205	60000.00
Anjali	Mechanical Keyboard	3500.00	2026-08-06	2	206	7000.00
Vikas	JBL Speaker	5000.00	2026-08-07	3	207	15000.00
Neha	Samsung Mobile	45000.00	2026-08-08	1	208	45000.00
Karan	Sony Camera	65000.00	2026-08-09	1	209	65000.00
Meena	Canon Printer	9000.00	2026-08-10	2	210	18000.00

10 rows in set (0.00 sec)

Q. Show the order with employee details
mysql> select e.emp_name,e.department,o.order_date,o.status from employees e inner join orders o on o.employee_id=e.employee_id;

Example: Find employee wise order count.
mysql> select e.emp_name,count(o.employee_id) from employees e inner join orders o on e.employee_id=o.employee_id group by e.employee_id;

Example: find the payment details with customer
mysql> select c.name,p.payment_method,p.amount,p.payment_date from customers c inner join orders o on o.customer_id=c.customer_id inner join payments p on p.order_id=o.order_id;

Self join: A self join is a join in which a table join with itself

a. It is used when rows in the same table have a relationship with each other
b. Often, aliases are used distinguish the two instance of the same table

Syntax(general)

Select a.column,b.column from tablename A join tablename B on a.columname=b.column_name

When self join is recommended

---

a. To compare rows within a same table
b. To find related record in the same table

Example: suppose consider we have employee table like as

Example: WA SQL Query find employees and their managers

Example: find the employee without manager

Q. Cartesian project or Cross Join?

A Cartesian product or cross join occurs when
Every row of one table is combined with every row of another table

a. If Table A has m rows and Table B has n rows , the resulting table will have m x n rows
b. No condition (ON or where) is required ,although you can filter result with where
c. Often used to generate combinations or base of other joins

Q. When is Cross join required?

---

1. You want to all possible combinations of rows from two tables
2. Generating test data or schedules where every option needs to pair with every other option
3. Creating combinations for reports or statistical analysis

Etc

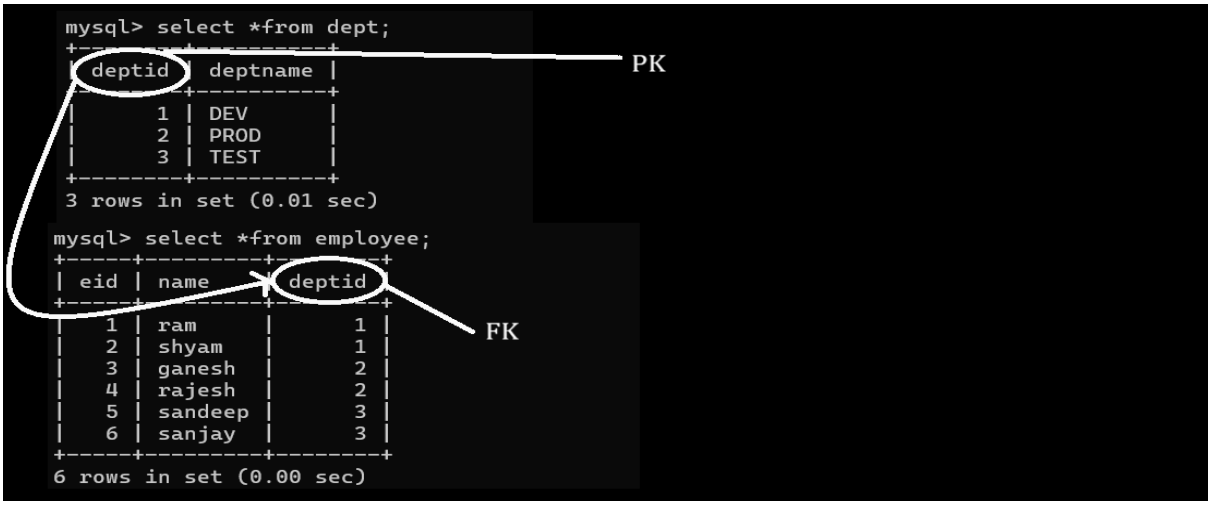
Association and keys and normalization

Association means relationship between two tables called as association

Types of association

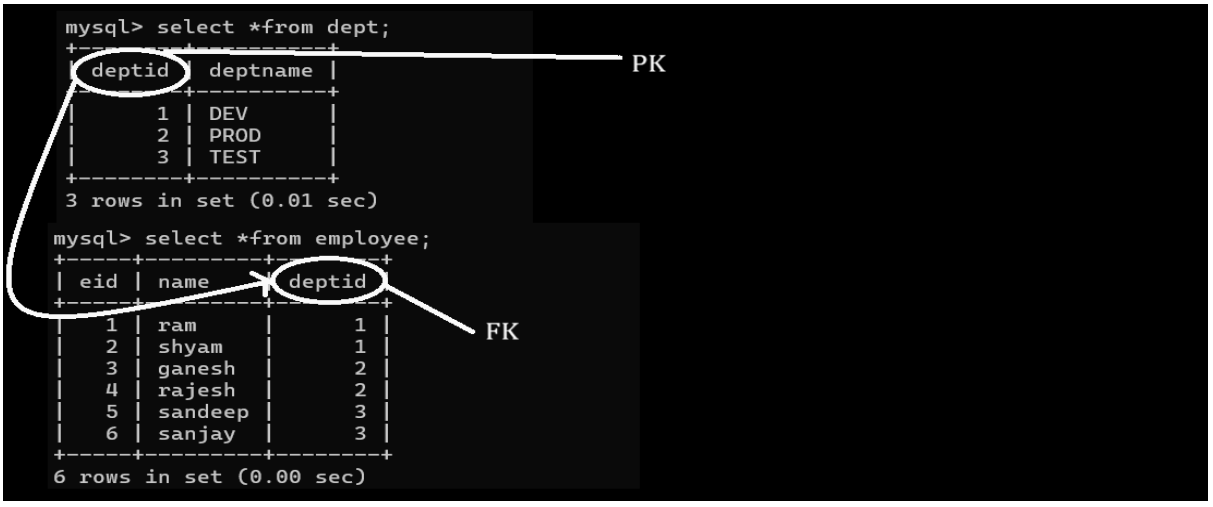
---

a. **One to many :** if one record of particular table is associated with more than one record in another table called as one to many association
   Note: practically if we want to one to many association we have to use primary and foreign key concept
   Example: Single dept can more than one employee



b. **Many to one : Many to one association is opposite of one to many**

Example: i can say single dept can have more than one employee so it is one to many but if say multiple can point to single dept called as many to one association



Note: inverse of one to many is many to one

c. **Many to many :** many to many is a combination of one to many and many to one

If we want to implement many to many practically we have to create one join table in which contain foreign keys

Example: suppose consider we are designing database of private training institute so there is chances single course can have more than one students and single student can join more than one courses

d. **One to one : one to one association single record of one table points to single record in another table**

Example: if we thinking about social media application we can say single user can have single profile

If we want to implement the association practically we need to know more detail about different types of keys and its important

1. **Primary:**primary key is by default unique + not null means we cannot duplicate primary key as well cannot store null values as well as cannot create more than column in same as primary key
   Note: refer constraints topic in notes

2. **Foreign:** foreign key means when we use primary key as reference in another table called as foreign key
   Note: primary and foreign key combination normally recommended to implement one to many or many to one association

3. **Unique :** unique key is type of key where column not allowed duplicated values but can allow null value and we can create more than one column as unique key in table

   Note: if we combine unique+ foreign key then we can implement the one to one association or relationship in table two table
   Foreign key is responsible provide relationship between tables and unique is responsible for avoid duplicate relationship in records
   When we avoid duplicate relationship in tables called as one to one association

   mysql> create table luser(userid int(5) primary key auto_increment,name varchar(200),email varchar(200),contact varchar(200));
   Query OK, 0 rows affected, 1 warning (0.09 sec)

   mysql> create table lprofile(pid int(5) primary key auto_increment,pname varchar(200),pphoto varchar(200),username varchar(200),password varchar(200),userid int(5) unique,foreign key(userid) references luser(userid))
   -> ;

```
mysql> select *from luser;
+--------+-------+-----------------+---------+
| userid | name  | email           | contact |
+--------+-------+-----------------+---------+
|      1 | ram   | ram@gmail.com   | 123456  |
|      2 | shyam | shyam@gmail.com | 4123456 |
+--------+-------+-----------------+---------+
2 rows in set (0.00 sec)

mysql> select *from lprofile;
+-----+-------+--------+----------+----------+--------+
| pid | pname | pphoto | username | password | userid |
+-----+-------+--------+----------+----------+--------+
|   1 | rama  | r.png  | rama     | ram      |      1 |
+-----+-------+--------+----------+----------+--------+
1 row in set (0.00 sec)
```

4. **Composite key (composite primary ,composite foreign):**
   Composite key means when we create more than one column for unique identity purpose called as composite key

Important points related with composite key

a. Composite key contain more than one column
b. All column should be duplicate
c. If any one column value is duplicated then it is allowed

Types of composite key

a. **Composite primary key :** composite primary key means if we create two or more than two column as primary key called as composite primary key

Q. What are the benefits of composite keys?

A composite primary key is used when:

a. You need to ensure uniqueness based on multiple fields
b. A single column is not enough to identify unique record
c. There is many to many relationship

How to create composite primary key

```
mysql> create table employee(eid int(5) ,ename varchar(200),email
varchar(200),contact varchar(200),username varchar(200),password
varchar(200),primary key(eid,email));
Query OK, 0 rows affected, 1 warning (0.07 sec)
```

b. **Composite foreign key: composite foreign key works**

```
mysql> create table vendor(vid int(5) primary key auto_increment,vname varchar(200));
Query OK, 0 rows affected, 1 warning (0.06 sec)

mysql> create table customer(cid int(5) primary key auto_increment,cname varchar(200));
Query OK, 0 rows affected, 1 warning (0.05 sec)
mysql> create table vcjoin(vid int(5), foreign key(vid) references vendor(vid),cid int(5),foreign
key(cid) references customer(cid),unique(vid,cid));
Query OK, 0 rows affected, 2 warnings (0.14 sec)
```

c. **Composite unique :** when we create two column as unique called as composite unique key

```
mysql> create table book(bookid int(5) primary key,bookname varchar(200),isbn
varchar(200),unique(bookid,isbn));
Query OK, 0 rows affected, 1 warning (0.16 sec)
```

5. **Candidate key :** A candidate key is any column or set columns that can uniquely identify a row in a table called as candidate key
   a. A table can have multiple candidate keys
   b. Only one candidate key becomes the primary key
   c. Rest all are the alternative keys

Example of candidate keys

Consider a student table

Note: you can create candidate by using unique keyword or primary key keyword

Q. What is the difference between composite primary key and candidate key?

feature	**Composite primary key**	**Candidate key**
Definition	Composite primary key is formed by 2+ columns	Any column that can uniquely identify record
uniqueness	Ensure uniqueness using combining fields	Any unique field i.e single column or multiple column
One vs many	Only one composite primary key allowed in table	A table can have multiple candidate keys
example	Primary key(sid,nic)	Sid primary key Nic unique Email unique

6. **Super key :** super key is a set of one or more column that uniquely identify a column called as super key
    a. It may contain extra unnecessary attributes
    b. Every candidate key is super key but not every super key is a candidate key

Possible super keys

---

a. **{StudentId}**
b. **{Email}**
c. **{NIC}**
d. **{studentId,Name}**

7. **Surrogate key:** surrogate key means key automatically generate primary key or unique key

Auto_increment

Normalization concept

Note: if we want to work with normalization we need to know the following concept

a. **Dependencies concept in database**

Dependencies in a database table refers to relationship between attributes or column where the value of attribute depends on the value of another attribute

Suppose consider we have employee table and it contain two employee name same i.e ram and we have question we want to increment salary of ram so the confusion is we have two ram how we can identify uniquely record of ram for that purpose we have use empid for identify ram because empid is primary key so empid is not duplicate so name column identify is dependent on empid as well as salary column identify is dependent on empid so we can say one column identify is dependent on another column called as dependencies

Types of dependencies

1. **Functional dependencies :** A Functional dependency (FD) in a database occur when the value of one attribute or set of attribute uniquely determines the value of another attribute called as functional dependencies
   Written as:
   A ->BA or BA ->A

Suppose consider we have student table

Student

StudentID	Name	Department	Email
101	ABC	CSE	abc@gmail.com
102	MNO	IT	mno@gmail.com
103	PQR	CSE	pqr@gmail.com

Functional dependencies
StudentID -> Name
StudentId -> Department
StudentId ->Email
Email -> Name

2. **Partial dependency :** Partial dependency occurs in a database when non primary attribute depend on part of composite primary key or not whole the key or part of the candidate key
   If your table contain partial dependency then table not in 2nd normal form

If we think about above database we have three tables like as student,course and score
If we think about score table it contain composite ky sid and cid and two non primary key attribute name as omarks and teachername
If we think about omarks column it is non primary attribute column but if we think about this column there is some question
Suppose if we think 80 marks obtain
There is two question

a. We need to know student name as well as course name
   So you can identify student using sid from composite key and course using cid from composite key means sid and cid is combination which help us to identify obtain marks means here obtain non primary key column identify is dependent on sid and cid so this column is 100% dependent on composite key so this column not generate the partial dependency in table
b. If we think about teachername column and if we want to identify subject of teacher e,g if we think about Mr.x so we need to course name of Mr.x so uniquely identify teacher we need to its cid not need to know sid here teachername column identity is only dependent on cid and cid is part of composite means teachername column identify

not 100% dependent on composite key it is dependent on half portion of the composite key so teachername column create partial dependency here

3. **Transitive dependency :** when one non primary key attribute is dependent on another non primary key attribute called as transitive dependency

If we think about above database we have score table which contain transitive dependency
If we think about omarks column it is dependent on sid,cid but there primary key columns as well as omarks column is dependent on examname , totalmarks also but these are the non primary key column so we can say examname and totalmarks these column create transitive dependency

Example: if we think about 80 marks obtain by
There some question

a. **Need to know student name :** you can identify student name by using sid
b. **Need to know course name : you can identify course name by using cid**
   Note: here cid and sid is composite key so these are not non primary attribute so here is no partial dependency
c. **Need to exam name :** if we want to identify exam name we have to have column name as examname and examname is non primary attribute
   So omarks is also non primary and exam name is also non primary attribute so here we have transitive dependency happen
d. **Need to know total marks:** if we want to identity total marks we have column name as totalmarks and it is also non primary attribute so here omarks is also non primary and totalmarks also non primary key attribute here these two columns create transitive dependency

Note: when your table contain transitive dependency then table not in third normal form

4. **Multivalued dependencies**

Normalization

Normalization is technique where we can decompose large database table into sub tables or smaller tables called as normalization

Q. What is the purpose of normalization?

---

a. **Avoid data duplication or redundancy**
b. **Avoid insertion/deletion/updation anomalies**

Insertion anomalies (confusion)

If we think about above database table shyam is a student not decide the branch so There are two chances

a. **Don't insert record of shyam in table**
b. **You have to store null value in dept and hod as well as fees**

Suppose consider scenario we want to find the admission count and we write query like as select count(aid) from admission here this query show the 2 records but actually fees paid by only one student it will create misunderstanding between account and management

Deletion anomalies

If we think about above table if we delete record using hod name as Mr.X
Then all record of CSE and student of CSE get deleted the meaning is dept is dependent on HOD it is not possible real time hod can replace

Update anomalies: update anomalies means there is possibility single SQL query may be update bunch of records at time unnecessary

Types of normalization

a. 1st ND
b. 2nd NF
c. 3rd NF
d. BCNF

e. 4th NF

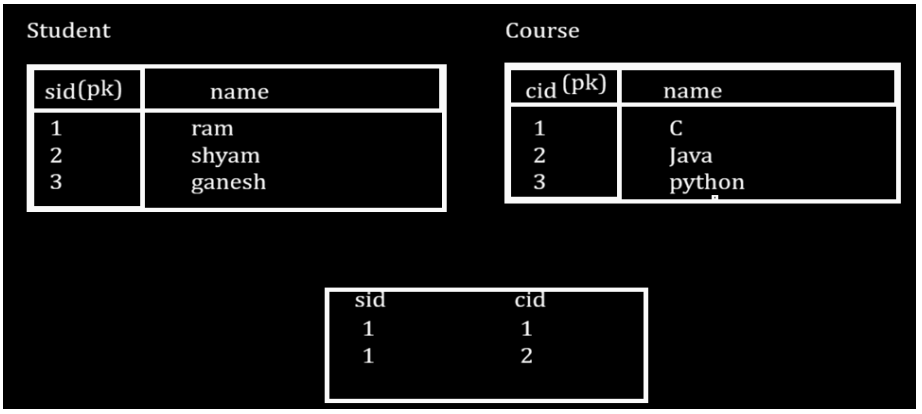
1st NF: table present in first normal form when table satisfy the following rules

a. Column must be contain atomic value
b. Column data type column data must be same
c. Column name should not duplicate in table

Example:

If we think about admission table this is not first normal form because course column not satisfy the first normal we store multiple values in it separating by comma
If you want to arrange table in first normal then your table look like as

If we think about above table it is in first normal form but there redundancy in table
You can divide the this table in smaller parts and provide relationship between them here course and student has many to many association database look like



2nd normal form:

Rules of second normal form

a. Table should be first normal form
b. Table should not have partial dependency

Note: if we think about score table here score table not in second normal form because here teachername column create partial dependency

Important point: refer partial dependency topic mention above for depth

If you want to arrange your score table in second normal form we have to remove partial dependency from table

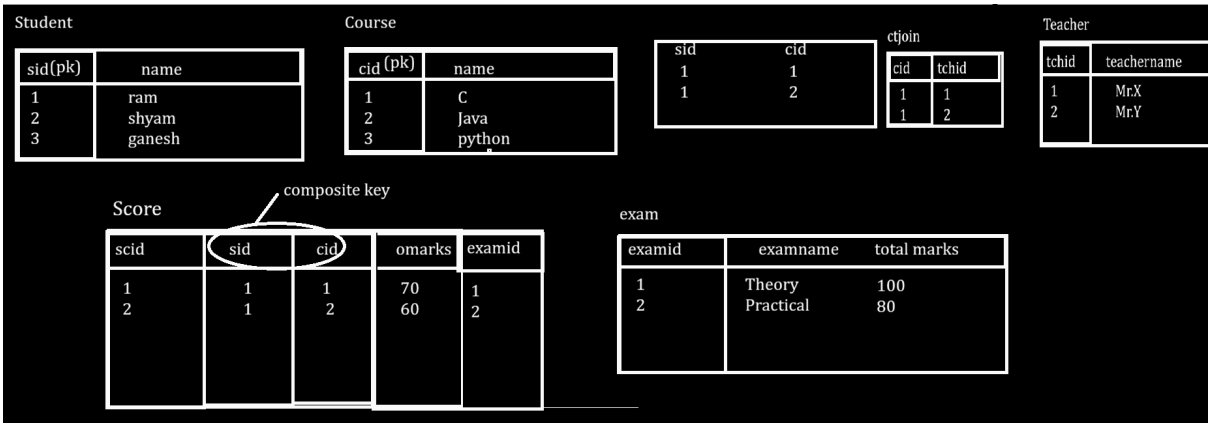
Note: if we think about above diagram score table in second normal form because there is no partial dependency

Third normal format

Rules of third normal form

a. Table must be second normal form
b. Table should not have transitive dependency

Note: if we think about above table it contain transitive dependency and examname and total marks column create transitive dependency
Important point: reference transitive dependency concept mentioned above
If we want to arrange this table in third normal format we have remove transitive dependency from table



If we think about above table there is no transitive dependency in table so it is in third normal form

SubQuery or Nested Query : SubQuery means query within query called as subquery
Subquery contains two categories **1. Outer query 2. Inner query**

Note: There is no standard syntax of subquery

Generalize syntax: select outerquery conjunction (inner query)
Note: conjunction are the some operation which is responsible connect inner query with outer query or provide inner query result with outer query etc

Normal conjunction/Operator used in sub query

---

1. < , > ,= ,!=, etc : normally this operator used in subquery when inner query generate single value as result
2. IN, exists ,any,all : these operators normally help us when your inner query generate more than value as result

Note: if we think about subquery first inner query get executed and its result pass to outer query

```
mysql> create table project(pid int(5) primary key auto_increment,projname
varchar(200),location varchar(200));
Query OK, 0 rows affected, 1 warning (0.08 sec)

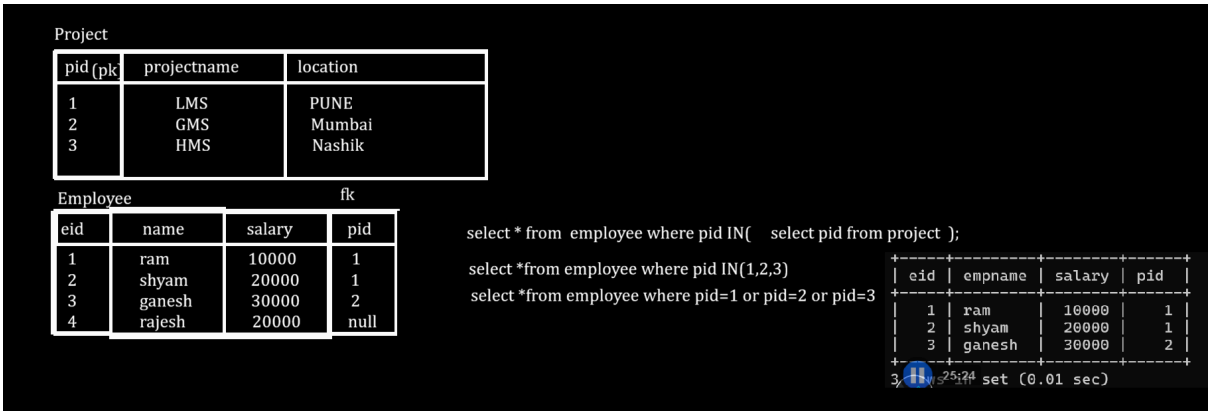
mysql> create table employee(eid int(5) primary key auto_increment,empname
varchar(2000),salary int(5),pid int(5),foreign key(pid) references project(pid));
Query OK, 0 rows affected, 3 warnings (0.09 sec)
```

There are two types of inner query

---

a. **Nested query or inner query:** inner query or nested query means where inner query execute first and its result used by outer query called as nested query

   Example: find employee who working on at one project



b. **Co-related subquery :** co related subquery means reference of outer query used in inner query normally for writing this type of query we use the exists operator
   Syntax: select *from tablename ref where exists (inner query)

Example: Find the employee who is not working on any project

Q. What is the difference between IN and an existing operator?

a. IN operator execute inner query and pass its output as input to outer query
b. IN operator cannot work with null value but exists can work with null value
c. IN operator can use for avoiding multiple or and exists check the value present and exists operator return boolean value i.e either true or false

Example: you can subquery in insert command

```
mysql> insert into employee values('0','ganesh',20000,1,(select curdate() ));
Query OK, 1 row affected (0.01 sec)
```


How use any and all operator in subquery

The ANY and ALL operators allow you to perform a comparison between a single column value and a range of other values

The ANY Operator

The Any Operator

a. Return a boolean value as a result
b. Return true if any of the subquery values meet the condition

ANY means that the condition will be true if the operation is true for any of the values in the range
Note: The operator must be a standard comparison operator(= , <>,!=,>, <,<=,>= etc)

Example: WA SQL Query to fetch any employee data works in project 1 or location mumbai

ALL Operator:

1. **Return a boolean value as result**
2. **Return True if all of the subquery values meet the condition**
3. **We can use with select , where and having also**

Views in SQL

Q. What are Views?

A SQL view is a virtual table created from the result of a select query. It does not store data physically but displays data stored in the underlying table.

Q. Why use views?

---

1. **Provide table security:** if we use the views then we can give access to a particular column not the whole table.
   Means we can apply multiple view single tables and give restricted access of single tables to multiple users.

2. **Hide the complex queries :** suppose if we have complicated joins or subqueries and if we want to use it multiple times so writing the whole query is not a good approach so better way you can store query in views and execute the views so we get output of query.

How to create view practically in MYSQL

Syntax: create view viewname as select column1,column2…..columnn from tablename;
mysql> create view empview as select eid,empname from employee;

View with join

In the given example we hide the join in view.
Example with source code
mysql> create view cquery as select p.projname,p.location,e.empname,e.salary,e.jdate from project p inner join employee e on p.pid=e.pid;

How to delete view

If we want to delete view we have to use drop command
Syntax: drop view viewname;

How to update the view using MYSQL

If we want to update the view using mysql we have following command

Syntax: create or replace view viewname as select query

Note: create or replace command create view if not exist in database table and replace view if exist in database table.

mysql> create or replace view test as select salary from employee;
Query OK, 0 rows affected (0.06 sec)
Or

```
mysql> create or replace view empview as select eid,empname,salary from employee;
Query OK, 0 rows affected (0.35 sec)

mysql> select *from empview;
+-----+---------+--------+
| eid | empname | salary |
+-----+---------+--------+
|   2 | balu    |  20000 |
|   3 | c       |  30000 |
+-----+---------+--------+
2 rows in set (0.01 sec)
```

Rules to update view
Certain conditions not satisfy to update view

a. The select statement which is used to create view should not include group by clause or order by clause
b. Select statement should not have distinct keyword

c. The view should all not null values
d. View should not created using a nested query or complex queries

Etc

Procedure in SQL

Q. What is the procedure?

Procedure is a same like as function in java which is used for reuse SQL code
Or procedure is a block of SQL statement which is used for reuse SQL code
Called as procedure.

How to create procedure in MYSQL

Steps to create procedure in MYSQL

a. **Create procedure** : if you want to create procedure we have to use following syntax

```
delimiter character
create procedure procedurename(IN | OUT | INOUT  variablename datatype(size) .....)
begin

   write here your logics

end
delimiter
```

Q. What is the delimiter?

IN SQL delimiter indicate end of SQL statement character called as delimiter and by default it is semi colon but when we work with a procedure, trigger or function
Then your delimiter should not semicolon

Important point: when we work with procedure then we have there are three types of parameter in procedure

IN : this parameter is used for accept input value as parameter from procedure calling point

OUT : this parameter is used for generating output as parameter and sending the result at the procedure calling point.

IN OUT : this parameter work as input parameter as well as output parameter

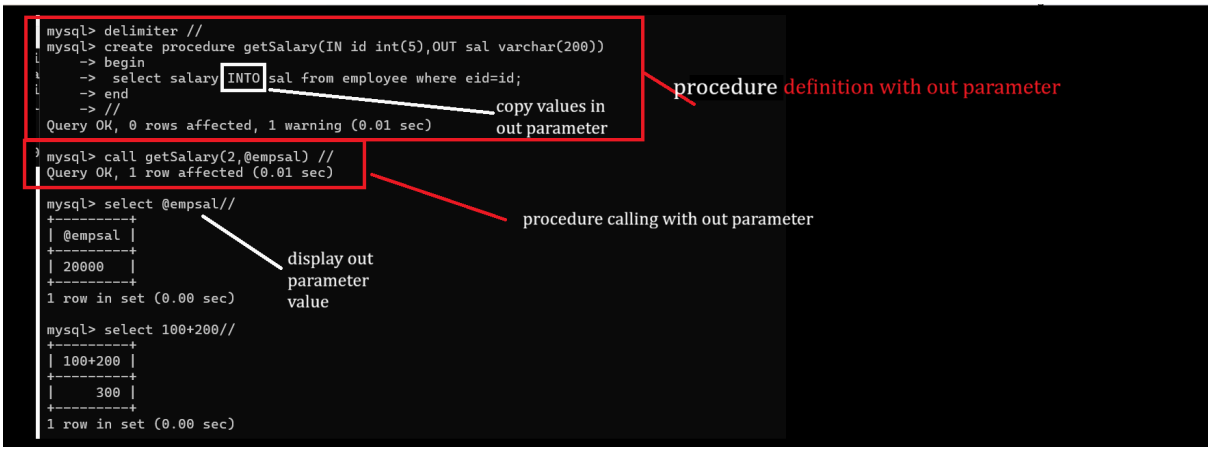
Example: WAP to create procedure to store data in employee table using a procedure

b. **Call procedure: if you want to reuse procedure we have to call it**
Syntax: call procedurename(parameters)

How to get out parameters from procedure

Example: Create procedure which return salary of employee by using its id
Means you have pass id as input parameter and return salary as output parameter
Steps.

a. **Create procedure and mark parameter with OUT keyword**
b. **Store return value in OUT parameter using INTO keyword**
c. **Catch out parameter in procedure calling point**



Index in SQL

Index is a special lookup table that are used by the database searching engine to accelerate data retrieval or improvise the query performance

Q. Why are indexes in SQL used?

---

a. **Improved query performance :** The primary reason for using indexes is to accelerate query processing. Indexes can drastically reduce the amount of data the server needs to examine

b. **Efficient data access :** indexes provide a quick way to access row data for select statements. This is particularly beneficial for table with large amount of data

c. **Sorting and Grouping speed :** Indexes improved the speed of data retrieval with order by clause or group by clause etc
d. **Optimized join operations:** In databases with multiple tables ,indexes improve the speed of join operations by quickly locating the joining rows in each table.

Limitation of indexes

Overuse of indexes: while indexes speed up data retrieval , they can slow down data input through insert/update/delete statements
Each index needs to be updated when data in modified
Storage: indexes consume additional disk space
Maintenance overhead: Indexes need to be maintained and rebuilt overtime , which can add, overhead to database maintenance

Types of indexes in SQL

There are two types of indexes in SQL

a. **Primary/cluster index:** primary key use by default primary index or cluster index and it is combination on non null and unique
b. **Secondary index /non cluster index :** when we apply unique constraints on database columns or create user defined index using a create index command by developer called as secondary index.

How to create index practically using MYSQL

Syntax: create index indexname on tablename(columname)

How to drop the index: if we want to drop the index we have command
Syntax: drop index indexname on tablename;
Example:

How to declare variable in SQL

There are two ways to declare variable

a. **Declare variable as system variable**

Syntax: set @variablename=value

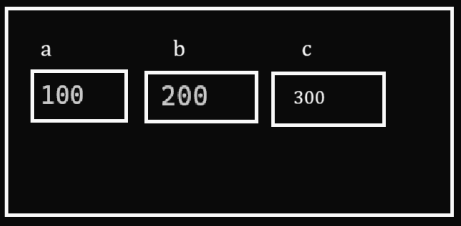
b. **Declare variable in user define block like as procedure, trigger, function etc**

If we want to declare variable in user define block then we have use following syntax

Syntax: declare variablename as datatype

```
mysql> create procedure calculateAdd()
    -> begin
    -> DECLARE a INT;
    -> DECLARE b INT;
    -> DECLARE c INT;
    -> SET a=100;
    -> SET b=200;
    -> SET c=a+b;
    -> select c;
    -> end
    -> //
Query OK, 0 rows affected (0.03 sec)

mysql>
```



How to write if else block in SQL

Example: Create Procedure using MYSQL pass number as parameter and check number is even or odd.

Example: Create procedure pass three parameter to procedure and check which is greater

How to write switch case in MYSQL

If we want to write switch case in mysql we have to use following syntax

```
CASE
  when condition THEN
     logic
 when  condition THEN
   logic
  when condition THEN
   logic
  ELSE
    logic
 END
```

Example: WAP to create procedure pass three parameter a,b,choice and when case 1 then calculate addition of two values, case 2 calculate multiplication of 2 values etc

How to write loop in MYSQL

```
DECLARE variablename datatype;

SET variablename=initialization;

label : LOOP
  SET increment or decrement
  write here logics
  IF condition THEN
    leave label;
  END IF;
END LOOP;
```

Example: Create procedure to display good morning message five times.

```
DECLARE i INT;
SET i=1;

loop_test:LOOP:
  select "good morning" as "message";
  SET i=i+1;
  IF i==5 THEN
    leave test_loop;
  END IF;
END LOOP;
```

How to write function in MYSQL

Function is a reusable block of SQL logics like as procedure the difference is procedure cannot return value but function can return value

Syntax

```
delimiter character
create function functionname(parameter1,parameter2....parameter..n) returns
datatype[characteristic]
begin
   write here logics
         return value;
end
```

Description about Characteristics of function

Characteristics decide the behaviour of function

When we create function then at least one of the characteristic should be use {DETERMINISTIC, NO SQL , READS SQL DATA} we have to specify in declaration

Q. What is DETERMINISTIC?

In MYSQL a deterministic function is a function that always returns the same result when given the same input values.

```
mysql> select getSquare(5)//
+--------------+
| getSquare(5) |
+--------------+
|           25 |
+--------------+
1 row in set (0.01 sec)
```

NOT DETERMINISTIC : The function may return different results even with the same input. This is used with function like as RAND(),NOW() or UUID() etc

NO SQL : the function does not execute any SQL Statement

READS SQL DATA: this feature or characteristic normally used when you want to fetch data from database or select query in function

Example: create function to return salary of employee by using its id.

```
mysql> delimiter //
mysql> create function getSalaryByUsingId(id int(5))returns INT READS SQL DATA
    -> begin
    -> DECLARE empSalary INT;
    -> select salary INTO empSalary from employee where eid=id;
    -> return empSalary;
    -> end
    -> //
Query OK, 0 rows affected, 1 warning (0.02 sec)
```

Q. What is the difference between function and procedure?

1. Function mainly used for performing calculations and returning a computed value such as calculating GST ,finding grade or computing annual salary etc
   A procedure is used to perform business operation like as inserting ,updating , deleting records or execute multiple SQL operations
2. A function must return exactly one value using the Return statement. A procedure does not have return a value but procedure can return value at calling using out parameter
3. Function can call in DML Statement and DQL statement not need to use call keyword but procedure can call by using call keyword only
   Example of function calling : select curdate(), select getEmployeeSalaryById(2);
   Example of procedure calling : call getEmployeeSalaryById(2,@Salary);
4. Function can support only IN parameters , meaning it accepts input values only. A procedure can support IN parameter as well as OUT and INOUT parameters

Trigger concept in SQL:

A Trigger in SQL is a special stored procedure that runs automatically when an INSERT/UPDATE or DELETE operation occurs on the table.

How to create trigger in MYSQL

Syntax:

```
delimiter character;
create trigger triggername [ before | after ]
{insert | delete | update } on tablename for each row
begin
   write here logics

end
```

There are six types of trigger we have

Before insert : before insert means trigger executed before inserting record in table on which we apply trigger or execute trigger

After insert : after insert means trigger executed after insertion record in table on which we apply trigger or execute trigger.

Before update : before update means trigger execute before update record in table on which we apply trigger or execute trigger.

After update : after update means trigger executed after update record in table on which we apply trigger or execute trigger.

Before Delete:before delete before delete means trigger execute before delete record in table on which we apply trigger or execute trigger.

After Delete: after delete means trigger executed after delete record in table on which we apply trigger or execute trigger.

Example: we want to create mini statement or transaction record automatically

```
mysql>
mysql> create table account(accid int(5) primary key auto_increment,accname
varchar(200),accbalance int(5));
   -> //
Query OK, 0 rows affected, 2 warnings (0.27 sec)

mysql> delimiter ;
mysql> create table minis(trid int(5) primary key auto_increment,accid int(5),foreign
key(accid) references account(accid),prevbal int(5),nextbal int(5),ttype varchar(200));
Query OK, 0 rows affected, 4 warnings (0.11 sec)

mysql> delimiter //
mysql> create trigger mmstmt after update on account for each row
   -> begin
   ->  IF OLD.accbalance < NEW.accbalance THEN
   ->    insert into minis
values('0',OLD.accid,OLD.accbalance,NEW.accbalance,'DEPOSIT');
   -> ELSE
   ->    insert into minis
values('0',OLD.accid,OLD.accbalance,NEW.accbalance,'WITHDRAW');
   -> END IF;
   -> end
   -> //
Query OK, 0 rows affected (0.04 sec)
```

Example: Create trigger when account delete then its mini statement should be delete

```
mysql> delimiter  //
mysql> create trigger dtras after delete on account for each row
   -> begin
   -> delete from minis where accid=OLD.accid;
   -> end
   -> //
Query OK, 0 rows affected (0.02 sec)
```

Cursor

A MYSQL cursor is a database object that enables the end user to retrieve , process and scroll through rows of the result set one at a time
Cursor performs operations on one row at a time.

Cursor enables the user to fetch and process each row individually, allowing for detailed manipulation and analysis of data.

Steps to work with MYSQL Cursor

---

1. **Declare the cursor to initialize in the memory**
   Syntax: DECLARE cursorname CURSOR for select statement
2. **Open the cursor to allocate memory**
   Syntax: open cursorname;
3. **Fetch the cursor to retrieve data**
   Syntax: fetch cursor_name into variablelist;

Note: fetch keyword is used for retrieve data from cursor and store in variable

4. **Close the cursor**
   Syntax: close cursorname;

ACID properties: ACID stands for atomicity consistency isolation and durability called ACID properties.

Atomicity: Atomicity means that either the entire transaction takes place at once or doesn't happen at all. There is no midway i.e transaction do not occur partially
Each transaction is considered as one unit either run to completion or is not executed at all.
It involves the following two operation

a. **Abort :** if a transaction aborts , changes made to the database are not visible.
b. **Commit:** if a transaction commit changes are visible.

Atomicity is also known as the All or nothing rule

If the transaction fails after completion of T1 but before completion of T2 (say write(x) but before write (y)) then the amount has been deducted from X but not added to Y. This results in an inconsistent database state and transaction should abort and if we want to verify then sum of both account balance before and after transaction should be same if same then complete if not then abort transaction

Consistency: this means that integrity constraints must be maintained so the database is consistent before and after the transaction.
It refers to the correctness of a database.
Example: The total amount before and after the transaction must be maintained.
Total before T occurs = 500+200 =700
Total after T occurs = 400+300 = 700
Therefore the database is a consistence if we found mismatch before and after then database is not consistent

Isolation: this property ensures that multiple transactions can occur concurrently without leading to the inconsistency of the database state. Transactions occur independently without interrupt to each other.
Changes occurring in a particular transaction will not be visible to any other transaction that is written to memory or has been committed.

If we think about above example here we consider two transaction interrupt each other so there is possible inconsistency because there is no isolation

Durability: this property ensure that once the transaction has completed execution , the update and modifications to the database are stored in and written to disk and they persist even if system failure occur and are stored in non-volatile memory i.e hard disk

TCL
DCL
Union, intersection,differential operation in database